MATLAB Guide — Ders Özeti

Desmond J. Higham & Nicholas J. Higham, *MATLAB Guide* (2. baskı)

Bölüm 2'den Bölüm 8.2'nin sonuna kadar (kitap sayfa 23–110)

Sayfa sayfa, kitaptaki bütün örneklerle birlikte Türkçe özet

Bu özet nasıl okunur?

- Başlıklardaki **sayfa numaraları kitabın kendi sayfa numaralarıdır.** PDF dosyasında aynı sayfa yaklaşık **25 sayfa ileridedir** (örneğin kitap s. 23 = PDF s. 48).
- Kitaptaki **her örnek** numaralandırılmıştır: *Örnek 5.12* = Bölüm 5'in 12. örneği.
- `>>` ile başlayan satırlar komut satırına yazılan komutlardır, altındaki satırlar MATLAB'ın çıktısıdır. `>>` olmayan kod bir script ya da fonksiyon dosyasının içeriğidir.
- 🔁 **Java** notları, Java bilen biri için MATLAB'ın farklı davrandığı yerleri gösterir.
- ⚠️ işareti sık yapılan hataları ve tuzakları gösterir.
- Kitap MATLAB 7 ile yazılmıştır. Yeni sürümlerde birkaç çıktı biraz farklı görünebilir (bunlar yerinde belirtilmiştir).

İçindekiler

Bölüm	Konu	Kitap sayfası	Örnek sayısı
2	Temeller (Basics)	23–32	25
3	MATLAB'ın Ayırt Edici Özellikleri	33–35	8
4	Aritmetik	37–43	38
5	Matrisler	45–62	79
6	Operatörler ve Akış Kontrolü	63–73	48
7	M-Dosyaları	75–85	26
8.1	İki Boyutlu Grafikler	87–101	26
8.2	Üç Boyutlu Grafikler	101–110	12

Bölüm 2 — Temeller (Basics)

Bölüm iki kısımdan oluşur: **2.1 Etkileşim ve script dosyaları** (s. 23–24) ve **2.2 Diğer temel konular** (s. 24–32).

Sayfa 23 — 2.1 Etkileşim ve script dosyaları

MATLAB **etkileşimli** bir sistemdir: `>>` işaretinin yanına komut yazıp Enter'a basınca sonuç hemen gelir. En basit haliyle bir hesap makinesi gibi kullanılır.

Örnek 2.1 — Altın oran

```
>> (1+sqrt(5))/2
ans =
    1.6180
```

Örnek 2.2 — Çok küçük sayı, üstel gösterim

```
>> 2^(-53)
ans =
   1.1102e-016
```

`1.1102e-016` = 1.1102×10^{-16}.

- Sonuç bir değişkene atanmazsa MATLAB onu **`ans`** (answer) değişkenine koyar. `ans` her yeni atanmamış sonuçta yenilenir ve sonradan normal bir değişken gibi kullanılabilir.
- **Değişkenler önceden tanımlanmaz**, değer atandığı anda oluşur. 🔁 Java'daki `double x;` gibi bir bildirim yoktur.

Örnek 2.3 — Değişkene atama

```
>> x = sin(22)
x =
   -0.0089
```

22 **radyan** kabul edilir, derece değil.

Örnek 2.4 — Noktalı virgül çıktıyı gizler

```
>> y = 2*x + exp(-3)/(1+cos(.1));
```

`y` hesaplanır ama ekrana yazılmaz.

Örnek 2.5 — Virgülle aynı satırda birden fazla komut

```
>> x = 2, y = cos(.3), z = 3*x*y
x =
     2
y =
    0.9553
z =
    5.7320
```

Örnek 2.6 — Virgül ve noktalı virgül karışık

```
>> x = 5; y = cos(.5); z = x*y^2
z =
    3.8508
```

`;` ile biten komutların çıktısı gizlendi, sadece `z` göründü.

Sayfa 24 — Script dosyaları

- MATLAB **büyük-küçük harfe duyarlıdır**: `x` ile `X` farklı değişkenlerdir.
- Birbiriyle ilgili komutlar `.m` **uzantılı** bir metin dosyasına yazılabilir. Buna **script M-dosyası** denir.

Örnek 2.7 — `marks.m` script'i (sınav notlarını işler)

```
%MARKS
exmark = [12 0 5 28 87 3 56];
exsort = sort(exmark)
exmean = mean(exmark)
exmed  = median(exmark)
exstd  = std(exmark)
```

`%` yorum satırıdır. `sort` sıralar, `mean` ortalama, `median` ortanca, `std` standart sapma hesaplar.

Örnek 2.8 — Script'i çalıştırmak: dosyanın adını yazmak yeterli

```
>> marks
exsort =
     0     3     5    12    28    56    87
exmean =
   27.2857
exmed =
    12
exstd =
   32.8010
```

`marks` yazmak, dosyadaki komutları tek tek elle yazmakla **tamamen aynıdır**. Ortalama = 191/7 = 27.2857. Script'lerin ayrıntısı Bölüm 7'de.

- **Kitaptaki yazım kuralı:** `>>` ile başlayan kod komut satırına yazılmıştır ve hemen altında çıktısı vardır. `>>` olmayan kod bir script dosyasının (ya da bir parçasının) içeriğidir.
- MATLAB'dan çıkmak için `exit` veya `quit`.

2.2 Diğer temel konular

Örnek 2.9 — 3×3 rastgele matris

```
>> A = rand(3)
A =
    0.9501    0.4860    0.4565
    0.2311    0.8913    0.0185
    0.6068    0.7621    0.8214
```

Her eleman [0,1] aralığında düzgün dağılımdan bağımsız olarak seçilir.

Sayfa 25 — Matris tersi, yuvarlama hataları, `format`, `help`

Örnek 2.10 — Matrisin tersi

```
>> inv(A)
ans =
    1.6740   -0.1196   -0.9276
   -0.4165    1.1738    0.2050
   -0.8504   -1.0006    1.7125
```

Örnek 2.11 — Ters ile çarpım birim matris olmalı

```
>> ans*A
ans =
    1.0000    0.0000         0
    0.0000    1.0000         0
         0   -0.0000    1.0000
```

`0.0000` ve `-0.0000` görünen değerler **tam sıfır değildir**. MATLAB yaklaşık **16 basamak** hassasiyetle hesaplar ama varsayılan olarak **5 basamak** gösterir.

Örnek 2.12 — Bilimsel gösterimle gerçek değerleri görmek

```
>> format short e
>> ans
ans =
   1.0000e+000  1.1102e-016            0
   2.7756e-017  1.0000e+000            0
             0 -2.2204e-016  1.0000e+000
```

Sıfır sanılan değerler 10^{-16} mertebesinde: bunlar **yuvarlama hatalarıdır**. Varsayılan biçime dönmek için `format short` ya da sadece `format`. Kitaptaki bütün çıktılar boş satırları gizleyen `format compact` ile üretilmiştir.

Örnek 2.13 — Yardım almak: `help foo`, `foo` hakkında bilgi verir

```
>> help sqrt
 SQRT   Square root.
    SQRT(X) is the square root of the elements of X. Complex
    results are produced if X is not positive.

    See also sqrtm.

    Reference page in Help browser
       doc sqrt
```

Sayfa 26 — Format tablosu ve yardım sistemi

Tablo 2.1 — `10*exp(1)` farklı biçimlerde:

Komut	Çıktı
`format short`	27.1828
`format long`	27.18281828459045
`format short e`	2.7183e+001
`format long e`	2.718281828459045e+001
`format short g`	27.183
`format long g`	27.1828182845905
`format hex`	403b2ecd2dd96d44
`format bank`	27.18
`format rat`	2528/93

- `help` çıktısındaki `sqrtm` ve `doc sqrt` altı çizili **bağlantılardır** (hyperlink); tıklayınca ilgili sayfaya gider.
- `help` metinlerinde fonksiyon adları kolay görünsün diye **BÜYÜK harfle** yazılır, ama komutlar **küçük harfle** yazılmalıdır (`sqrt`). Kullanıcının yazdığı M-dosyalarının adları ise dosya adındaki harflerle aynı yazılmalıdır.
- Tek başına **`help`** → fonksiyon klasörlerinin listesini verir (Tablo 2.2). Yola eklenmiş kendi klasörlerin de görünür.
- **`help general`** gibi klasör adı verilirse → o klasördeki fonksiyonlar listelenir.
- **`help help`** → `help` komutunun kendi ayrıntıları.
- **Help browser** (Şekil 2.1) en kapsamlı dokümantasyondur: bütün fonksiyonlar, sürüm notları, html ve PDF kılavuzlar. İçinde içindekiler, dizin, arama ve demo sekmeleri olan bir gezinme paneli vardır. Masaüstündeki **"?"** simgesiyle, **Help** menüsünden ya da `helpbrowser` yazarak açılır.
- **`doc foo`** → `foo` 'nun yardımını Help browser'da açar.
- **`docsearch('phrase')`** → dokümantasyonda tam metin araması.
- **`helpwin`** → `help` klasör listesini Help browser'da açar; klasöre tıklayınca içindeki M-dosyaları görünür.
- **`lookfor keyword`** → adını bilmediğin fonksiyonu anahtar kelimeyle arar.

Örnek 2.14 — Anahtar kelimeyle arama

```
>> lookfor elliptic
ELLIPJ  Jacobi elliptic functions.
ELLIPKE Complete elliptic integral.
PDEPE   Solve initial-boundary value problems for parabolic-elliptic
        PDEs in 1-D.
```

Sayfa 27 — Tablo 2.2: MATLAB klasör yapısı (`>> help` çıktısı, Windows)

Klasör	İçerik
`matlab\general`	Genel amaçlı komutlar
`matlab\ops`	Operatörler ve özel karakterler
`matlab\lang`	Programlama dili yapıları
`matlab\elmat`	Temel matrisler ve matris işlemleri
`matlab\elfun`	Temel matematik fonksiyonları
`matlab\specfun`	Özel matematik fonksiyonları
`matlab\matfun`	Matris fonksiyonları, sayısal lineer cebir
`matlab\datafun`	Veri analizi ve Fourier dönüşümleri
`matlab\polyfun`	İnterpolasyon ve polinomlar
`matlab\funfun`	Fonksiyon fonksiyonları ve ODE çözücüler
`matlab\sparfun`	Seyrek (sparse) matrisler
`matlab\scribe`	Açıklama ve grafik düzenleme
`matlab\graph2d`	İki boyutlu grafikler
`matlab\graph3d`	Üç boyutlu grafikler
`matlab\specgraph`	Özel grafikler
`matlab\graphics`	Handle Graphics
`matlab\uitools`	Grafik kullanıcı arayüzü araçları
`matlab\strfun`	Karakter dizileri (string)
`matlab\imagesci`	Görüntü ve bilimsel veri giriş/çıkışı
`matlab\iofun`	Dosya giriş/çıkışı
`matlab\audiovideo`	Ses ve video desteği
`matlab\timefun`	Zaman ve tarih
`matlab\datatypes`	Veri tipleri ve yapılar (struct)
`matlab\verctrl`	Sürüm kontrolü
`matlab\codetools`	Kod oluşturma ve hata ayıklama komutları
`matlab\helptools`	Yardım komutları
`matlab\winfun`	Windows işletim sistemi arayüz dosyaları (COM/DDE)
`matlab\demos`	Örnekler ve demolar
`toolbox\local`	Tercihler
`MATLAB7\work`	(İçindekiler dosyası yok)

💡 Bir kütüphanenin rafları gibi düşün: `help` rafları, `help elfun` bir raftaki kitapları, `help sqrt` tek bir kitabı gösterir.

Sayfa 28 — Help browser ve komut satırında düzenleme

- **Şekil 2.1:** Help browser penceresi (solda gezinme paneli, sağda "Begin Here" sayfası).
- Yazım hatası ok tuşları ve Backspace/Delete ile düzeltilir.
- **Yukarı ok** önceki komutları, **aşağı ok** sonrakileri getirir. Birkaç harf yazıp yukarı oka basınca o harflerle başlayan son komut gelir.

- Emacs tarzı Ctrl kısayolları da desteklenir (Tablo 2.3).
- **Command History** penceresinde bu ve önceki oturumlardaki komutlar durur; çift tıklanan komut yeniden çalışır.
- **Tab ile tamamlama:** Fonksiyon/değişken adının ilk harflerini yazıp Tab'a basınca MATLAB tamamlar; birden fazla seçenek varsa liste sunar.
- **Shift+Enter:** Birden fazla satırı yazıp en sonda Enter ile hepsini birlikte çalıştırır.
- `clc` **:** Command Window'u temizler (değişkenler silinmez).
- **Ctrl+C:** Çalışan hesabı durdurur (gömülü bir fonksiyon çalışıyorsa tepki biraz gecikebilir).
- `...` **(üç nokta):** Satırın alt satırda devam ettiğini gösterir.

Örnek 2.15 — Satır devamı (sayfa 28–29)

```
>> x = 1 + 1/2 + 1/3 + 1/4 + 1/5 + ...
       1/6 + 1/7 + 1/8 + 1/9 + 1/10
x =
    2.9290
```

Sayfa 29 — Kısayollar, özel değerler, karmaşık sayılar

Tablo 2.3 — Komut satırı düzenleme tuşları ("Ctrl-p" = Ctrl'ye basılıyken p):

Tuş	Ctrl karşılığı	İşlem
Yukarı ok	Ctrl-p	Önceki satırı getir
Aşağı ok	Ctrl-n	Sonraki satırı getir
Sol ok	Ctrl-b	Bir karakter geri
Sağ ok	Ctrl-f	Bir karakter ileri
Ctrl + sol ok		Bir kelime sola
Ctrl + sağ ok		Bir kelime sağa
Home	Ctrl-a	Satır başı
End	Ctrl-e	Satır sonu
Esc	Ctrl-u	Satırı temizle
Del	Ctrl-d	İmleçteki karakteri sil
Backspace	Ctrl-h	Önceki karakteri sil
	Ctrl-k	İmleçten satır sonuna kadar sil
Insert		Ekleme modunu aç/kapat
Shift-Home		Satır başına kadar seç
Shift-End		Satır sonuna kadar seç

- Örnek 2.15'teki x değeri şunu gösterir: MATLAB'da **tamsayılarla aritmetik de ondalıklı (floating point) yapılır**. 🔁 Java'da `1/2` = 0 olur; MATLAB'da `1/2` = 0.5.

Özel değerler: - `pi` = π = 3.14159... - `i` ve `j` = sanal birim $\sqrt{-1}$. Karmaşık sayı yazma yolları: `2-3i`, `2-3*i`, `2-3*sqrt(-1)`, `complex(2,-3)`.

⚠️ `i` bir değişken olarak kullanılmışsa (örneğin döngü sayacı) `2-3*i` yanlış sonuç verir. Bu yüzden `2-3i` ya da `complex(2,-3)` tercih edilir. Genel kural: var olan fonksiyon ve değişken adlarıyla yeni değişken oluşturmaktan kaçın.

Karmaşık sayı fonksiyonları: `conj` (eşlenik), `real` (gerçek kısım), `imag` (sanal kısım).

Örnek 2.16

```
>> w = (-1)^0.25
w =
   0.7071 + 0.7071i
```

Örnek 2.17

```
>> z = conj(w)
z =
   0.7071 - 0.7071i
```

Sayfa 30 — Workspace ve değişken yönetimi

Örnek 2.18

```
>> [real(z) imag(z)]
ans =
    0.7071   -0.7071
```

Örnek 2.19 — Euler formülü e^(iπ) = −1

```
>> exp(i*pi)
ans =
  -1.0000 + 0.0000i
```

Örnek 2.20 — `who` sadece adları, `whos` boyut, bellek ve tipi de verir

```
>> whos
  Name        Size      Bytes  Class
  A           3x3          72  double array
  ans         1x1          16  double array (complex)
  exmark      1x7          56  double array
  exmean      1x1           8  double array
  exmed       1x1           8  double array
  exsort      1x7          56  double array
  exstd       1x1           8  double array
  w           1x1          16  double array (complex)
  x           1x1           8  double array
  y           1x1           8  double array
  z           1x1          16  double array (complex)

Grand total is 31 elements using 272 bytes
```

Tek bir sayı bile **1×1'lik matristir**. Her double 8 byte; karmaşık sayılar 16 byte (gerçek + sanal).

- **`clear var`** → `var` değişkenini siler. **`clear`** → bütün değişkenleri siler.
- **`workspace`** → Workspace browser'ı açar (Şekil 2.2). Desktop → Workspace menüsünden de açılır.
- Bir değişkene çift tıklamak ya da **`openvar('A')`** → Excel benzeri **Array Editor**'da açar (Şekil 2.3).
- **Değişken adları:** harfle başlar; harf, rakam ve alt çizgi içerebilir; büyük-küçük harfe duyarlıdır; en fazla `namelengthmax` karakter.

Örnek 2.21

```
>> namelengthmax
ans =
    63
```

Kaydetme ve yükleme: - **`save filename`** → bütün değişkenleri `filename.mat` dosyasına kaydeder (File → Save Workspace As menüsü de olur). - **Örnek 2.22:** `save filename A x` → sadece `A` ve `x` kaydedilir. - `-v6` eklenirse dosya MATLAB 6 ile de okunabilir. - **`load filename`** → değişkenleri geri yükler; tek tek değişkenler `save` ile aynı sözdizimiyle yüklenebilir.

Sayfa 31 — Şekiller, `diary`, `disp`

- **Şekil 2.2:** Workspace browser (ad, değer, boyut, sınıf sütunları). **Şekil 2.3:** Array Editor'da `A` matrisi.
- `save` / `load` varsayılan olarak **ikili (binary)** biçimde çalışır; seçenekle ASCII de seçilebilir. MAT dosyaları farklı bilgisayarlardaki MATLAB'lar arasında taşınabilir.
- **`uiimport`** (ya da File → Import Data) → veri almak için grafik sihirbaz (Import Wizard).
- **`diary`** → oturumu rapor için dosyaya kaydeder:
- `diary filename` → bundan sonraki girdi ve (çoğu) metin çıktısı dosyaya yazılır
- `diary off` → kaydı durdurur; `diary on` → aynı dosyanın sonuna eklemeye devam eder
- **`disp`** → değişkenin adını yazmadan sadece değerini gösterir.

Örnek 2.23

```
>> A = eye(2); disp(A)
     1     0
     0     1
```

Sayfa 32 — İşletim sistemi ve bilgi komutları

Örnek 2.24 — Metin ve sayı yazdırmak

```
>> disp('Result:'), disp(1/7)
Result:
    0.1429
```

İşletim sistemi komutları: `cd` (klasör değiştir), `copyfile` (dosya kopyala), `mkdir` (klasör oluştur), `pwd` (bulunduğun klasör), `dir` veya `ls` (klasör içeriği), `delete` (dosya sil).

Örnek 2.25 — Başına `!` konan komut doğrudan işletim sistemine gider: `!emacs myscript.m` (Emacs editörüyle dosyayı açar).

Tablo 2.4 — Bilgi ve demo komutları:

Komut	İşlevi
`bench`	Bilgisayarın hızını test eden kıyaslamalar
`demo`	Demo koleksiyonu
`info`	The MathWorks iletişim bilgileri
`ver`	MATLAB ve araç kutularının sürüm numaraları ve tarihleri
`version`	MATLAB'ın sürüm numarası ve tarihi
`whatsnew`	Help browser'da sürüm notlarını açar; `whatsnew matlab` readme dosyasını, `whatsnew toolbox` ilgili araç kutusunun readme dosyasını gösterir

Bölüm sonu alıntıları: Beatles'ın "*Help!*" şarkısı (yardım komutlarına gönderme) ve "*If ifs and ans were pots and pans, there'd be no trade for tinkers*" atasözü (`if` ve `ans` üzerine kelime oyunu).

Bölüm 2 — Akılda kalması gerekenler

1. Atanmamış sonuç `ans`'a gider; `;` çıktıyı gizler, `,` aynı satırda komutları ayırır.
2. Değişkenler önceden tanımlanmaz; büyük-küçük harf fark eder.
3. `1/2` = 0.5 — bütün hesaplar ondalıklıdır.
4. Ekranda 5 basamak görünür ama hesap ~16 basamakla yapılır; küçük yuvarlama hataları normaldir.
5. Yardım: `help`, `doc`, `lookfor`. Değişkenler: `who`, `whos`, `clear`, `save`, `load`. Kayıt: `diary`. Yazdırma: `disp`.
6. `i` ve `j`'yi değişken adı olarak kullanma.

Bölüm 3 — MATLAB'ın Ayırt Edici Özellikleri

MATLAB'ı diğer modern programlama dillerinden ayıran **üç özellik** bu bölümde tanıtılır, ayrıntıları ileriki bölümlerdedir: (1) otomatik bellek ayırma, (2) değişken sayıda argüman, (3) karmaşık diziler.

Sayfa 33 — 3.1 Otomatik bellek ayırma

Değişkenler gibi **diziler de önceden tanımlanmaz**. Ayrıca MATLAB, bir atamanın anlamlı olması için gerekiyorsa **dizinin boyutunu kendiliğinden büyütür**.

Örnek 3.1 — Olmayan bir diziye değer atamak (çalışma alanı boşken)

```
>> x(3) = 0
x =
     0     0     0
```

`x` yokken 1×3 dizi oluşturuldu; boş kalan yerler **0** ile dolduruldu.

Örnek 3.2 — Diziyi büyütmek

```
>> x(6) = 0
x =
     0     0     0     0     0     0
```

Dizi 3'ten 6 elemana kendiliğinden büyüdü.

🔁 **Java:** `int[] x = new int[3]; x[5] = 0;` → `ArrayIndexOutOfBoundsException`. Java'da dizi boyutu sabittir.

Sayfa 33–34 — 3.2 Değişken sayıda argüman

- MATLAB fonksiyonları **sıfır veya daha fazla girdi** alır, **sıfır veya daha fazla çıktı** döndürür.
- **Girdiler** fonksiyon adının **sağında**, `( )` içinde; **çıktılar solunda**, `[ ]` içinde yazılır: `[çıktı1, çıktı2] = fonksiyon(girdi1, girdi2)`.
- Her çağrıda bütün argümanları vermek gerekmez; fonksiyon, verilen argümanların **sayısına ve türüne göre farklı davranabilir**.

`x = [3 4];` için:

Örnek 3.3 — `norm`, tek girdi: Öklid normu (2-norm)

```
>> x = [3 4];
>> norm(x)
ans =
     5
```

$\sqrt{(3^2 + 4^2)} = \sqrt{25} = 5$.

Örnek 3.4 — `norm`, iki girdi: 1-norm

```
>> norm(x,1)
ans =
     7
```

|3| + |4| = 7. İkinci girdi verilmezse **2** kabul edilir: `norm(x)` = `norm(x,2)`.

Örnek 3.5 — `max`, tek çıktı: en büyük eleman

```
>> m = max(x)
m =
     4
```

Örnek 3.6 — `max`, iki çıktı: en büyük eleman ve **indeksi**

```
>> [m,k] = max(x)
m =
     4
k =
     2
```

4, dizinin 2. elemanı (MATLAB'da indeksler **1'den** başlar).

Örnek 3.7 — `size`, tek çıktı

```
>> A = rand(5,3);
>> s = size(A)
s =
     5     3
```

Tek çıktıda `size` 1×2 vektör döndürür: [satır sayısı, sütun sayısı].

Örnek 3.8 — `size`, iki çıktı

```
>> [m,n] = size(A)
m =
     5
n =
     3
```

Satır ve sütun sayısı ayrı değişkenlere yazılır. (`m`'nin eski değeri 4'ün üzerine yazıldı.)

🔁 **Java:** Bir metot tek değer döndürür; birden fazla değer için dizi/nesne gerekir. MATLAB'da `[m,k] = max(x)` iki değeri doğrudan verir. Girdi sayısının değişmesi Java'daki **overloading**'e benzer.

Sayfa 34–35 — 3.3 Karmaşık diziler ve aritmetik

- MATLAB'ın temel veri tipi: **karmaşık sayılardan oluşan çok boyutlu dizi**; gerçek ve sanal kısımlar double hassasiyetle saklanır.
- **Matris** (2 boyutlu dizi), **vektör** ve **skaler** bu tipin özel halleridir. Tek bir sayı 1×1 matristir.
- Hesapların çoğu ondalıklı aritmetikle yapılır; veri karmaşıksa **karmaşık aritmetik otomatik** kullanılır.
- **Ayrı bir "gerçek sayı" tipi yoktur** (gerçek sayıların sanal kısmı saklanmaz).

Dil	Gerçek ve karmaşık sayılar
Fortran	Gerçek ve karmaşık için ayrı tipler
C, C++, Java	Sadece gerçek sayılar
MATLAB	Tek tip, ikisi de otomatik

- MATLAB'da **tamsayı tipleri** de vardır, ama esas amaçları hesap değil **bellek tasarrufudur** (Bölüm 4.4).

Sayfa 35 alıntıları (Cleve Moler): "*MATLAB'ın çekirdeği C ile yazılmıştır. MATLAB'ın büyük kısmı da MATLAB ile yazılmıştır, çünkü o bir programlama dilidir.*" (1999) — "*MATLAB, SPEAKEASY'ye ve daha az ölçüde APL'e benzer: hepsi tek satırlık komutları hemen işleyen etkileşimli dillerdir ve hepsinde temel veri tipi dizidir.*" (1982). Sayfa 36 boştur.

Bölüm 3 — Akılda kalması gerekenler

1. Diziler kendiliğinden oluşur ve büyür: `x(6) = 0` hata vermez.
2. Fonksiyonlar farklı sayıda girdi/çıktıyla çağrılabilir: `[m,k] = max(x)`.
3. Her şey matristir; karmaşık sayılar ek bir şey yapmadan çalışır.

Bölüm 4 — Aritmetik

Alt başlıklar: **4.1 IEEE aritmetiği** (s. 37–39), **4.2 İşlem önceliği** (s. 39–40), **4.3 Matematik fonksiyonları** (s. 39–40), **4.4 Diğer veri tipleri** (s. 40–43).

Sayfa 37 — 4.1 IEEE aritmetiği

MATLAB bütün hesaplarını varsayılan olarak **IEEE standardına uygun double hassasiyetli ondalıklı sayılarla** yapar (Java'daki `double` ile aynı standart).

Örnek 4.1 — MATLAB'ın çalıştığı bilgisayar türü

```
>> computer
ans =
PCWIN
```

Kitabın bilgisayarı Windows PC. Yeni sistemlerde `PCWIN64`, `MACA64` gibi değerler görülebilir.

double tipinin özellikleri: - Her sayı **64 bit** (8 byte) yer kaplar. - Sıfır olmayan sayılar yaklaşık **10^{-308} ile 10^{+308}** arasındadır. - **Unit roundoff** (birim yuvarlama) = $2^{-53} \approx 1.11 \times 10^{-16}$. Bir gerçek sayıyı kaydederken ya da toplama, çıkarma, çarpma, bölme ve karekök alırken oluşan **göreli hatanın üst sınırıdır**. - Kısaca: MATLAB yaklaşık **16 anlamlı basamak** doğrulukla saklar ve hesaplar.

Örnek 4.2 — `eps`: 1.0 ile ondan büyük bir sonraki ondalıklı sayı arasındaki mesafe

```
>> eps
ans =
   2.2204e-016
```

eps = 2^{-52} = unit roundoff'un **iki katı**.

Örnek 4.3 ve 4.4 — `eps(x)`: x'ten (büyüklük olarak) bir sonraki ondalıklı sayıya mesafe

```
>> eps(1/2)
ans =
   1.1102e-016

>> eps(2)
ans =
   4.4409e-016
```

x	eps(x)	Değer
1/2	1.1102e-16	2^{-53}
1	2.2204e-16	2^{-52}
2	4.4409e-16	2^{-51}

Sayı büyüdükçe saklanabilen sayılar arasındaki boşluk da büyür (sayı 2 katına çıkınca boşluk da 2 katına çıkar), göreli hata ise yaklaşık aynı kalır.

- IEEE standardı sayesinde **her hesap bir ondalıklı sayı üretir** (bazen özel türden). Sonuç `realmax`'tan büyükse **taşma (overflow)** olur ve sonuç `Inf` olur; `-realmax`'tan küçükse `-Inf` olur.

Sayfa 38 — Inf, NaN, realmin

Örnek 4.5 — En büyük double sayı

```
>> realmax
ans =
   1.7977e+308
```

Örnek 4.6 — Negatif taşma

```
>> -2*realmax
ans =
  -Inf
```

Örnek 4.7 — Pozitif taşma

```
>> 1.1*realmax
ans =
   Inf
```

Matematiksel olarak **tanımsız** bir hesap **NaN** (Not a Number) üretir: `0/0`, `inf/inf`, `0*inf` gibi.

Örnek 4.8

```
>> 0/0
Warning: Divide by zero.
ans =
   NaN
```

(Yeni sürümlerde uyarı çıkmayabilir.)

Örnek 4.9

```
>> inf/inf
ans =
   NaN
```

Örnek 4.10

```
>> inf-inf
ans =
   NaN
```

Bir NaN oluştuğunda **sonraki bütün hesaplara yayılır**:

Örnek 4.11

```
>> NaN-NaN
ans =
   NaN
```

Örnek 4.12

```
>> 0*NaN
ans =
   NaN
```

`realmin` → en küçük pozitif **normalleştirilmiş** sayı. `realmin`'den küçük bir sonuç ya **alttan taşar (underflow) ve sıfır olur** (`eps*realmin`'den küçükse) ya da **subnormal** bir sayı olur (mantisinin başında sıfır bitler olan, daha az hassas çok küçük sayı).

Örnek 4.13

```
>> realmin
ans =
   2.2251e-308
```

Sayfa 39 — Alttan taşma ve 4.2 İşlem önceliği

Örnek 4.14 — Subnormal sayı (saklanabilen en küçük pozitif sayı, 2^{-1074})

```
>> realmin*eps
ans =
   4.9407e-324
```

Örnek 4.15 — Alttan taşma: sonuç sıfıra yuvarlanır

```
>> realmin*eps/2
ans =
     0
```

Değer	Anlamı
`realmax` ≈ 1.8e+308	En büyük sayı; üstü `Inf`
`realmin` ≈ 2.2e-308	En küçük normal pozitif sayı
`realmin*eps` ≈ 4.9e-324	En küçük subnormal sayı; altı 0

Kitabın önerisi: Bu hesapları `format hex` yazdıktan sonra tekrarla; sayıların bellekteki ikili halini onaltılık biçimde görürsün.

4.2 İşlem önceliği

Aritmetik operatörler çoğu hesap makinesi ve programlama dilindeki öncelik kurallarına uyar (Tablo 4.1; bütün operatörler için Tablo 6.2). **Aynı seviyedekiler soldan sağa** hesaplanır. Parantez önceliği her zaman değiştirir ve karışıklığı önlemek için önerilir.

Örnek 4.16

```
>> 2^10/10
ans =
  102.4000
```

Önce üs: $2^{10} = 1024$, sonra bölme: 1024/10.

Örnek 4.17

```
>> 2 + 3*4
ans =
    14
```

Örnek 4.18

```
>> -2 - 3*4
ans =
   -14
```

Baştaki eksi **tekli eksidir** (seviye 2): (−2) − 12 = −14.

Örnek 4.19

```
>> 1 + 2/3*4
ans =
    3.6667
```

Bölme ve çarpma aynı seviyede, **soldan sağa**: (2/3)·4 = 2.6667, +1 = 3.6667.

Örnek 4.20

```
>> 1 + 2/(3*4)
ans =
    1.1667
```

Parantez önceliği değiştirdi: 2/12 = 0.1667, +1.

Örnek 4.21

```
>> [2^2^3 2^(2^3)]
ans =
    64   256
```

`2^2^3` soldan sağa: $(2^2)^3 = 64$. `2^(2^3)` = $2^8 = 256$. ⚠️ Matematikte 2^2^3 genelde 256 okunur; MATLAB 64 verir. **Parantez kullan.**

Sayfa 39–40 — Tablolar ve 4.3 Matematik fonksiyonları

Tablo 4.1 — Aritmetik operatör önceliği:

Seviye	Operatör
1 (en yüksek)	Üs alma `^`
2	Tekli artı `+`, tekli eksi `-`
3	Çarpma `*`, bölme `/`
4 (en düşük)	Toplama `+`, çıkarma `-`

4.3 Matematik fonksiyonları

Tam liste: `help elfun` ve `help specfun`. **Tablo 4.2** (`fun*` = adı "fun" ile başlayan birden fazla fonksiyon):

Fonksiyonlar	Kategori
`cos, sin, tan, csc, sec, cot` / `cosd, sind, tand, cscd, secd, cotd`	Trigonometrik
`acos, asin, atan, atan2, asec, acsc, acot` / `acosd, asind, atand, asecd, acscd, acotd`	Ters trigonometrik
`cosh, sinh, tanh, sech, csch, coth`	Hiperbolik
`acosh, asinh, atanh, asech, acsch, acoth`	Ters hiperbolik
`log, log2, log10, log1p, exp, expm1, pow2, nextpow2, nthroot`	Üstel
`ceil, fix, floor, round`	Yuvarlama
`abs, angle, conj, imag, real`	Karmaşık
`mod, rem, sign`	Kalan, işaret
`airy, bessel*, beta*, ellipj, ellipke, erf*, expint, gamma*, legendre, psi`	Özel matematik
`factor, gcd, isprime, lcm, primes, nchoosek, perms, rat, rats`	Sayılar teorisi
`cart2sph, cart2pol, pol2cart, sph2cart`	Koordinat dönüşümleri

- Trigonometrik fonksiyonlar **radyanla** çalışır; adı **"d"** ile biten sürümler **derece** alır: `sin(pi/2)` = `sind(90)` = 1.
- **`expm1`** ve **`log1p`**: |x| çok küçükken $e^x - 1$ ve log(1 + x) değerlerini **doğru** hesaplar; doğrudan hesapta oluşacak **iptal hatasını (cancellation)** önler. Örneğin `exp(1e-20)-1` = 0 verirken `expm1(1e-20)` = 1e-20 verir.
- Airy ve Bessel fonksiyonları, Amos'un Fortran kütüphanesine MEX arayüzüyle hesaplanır (kaynak: `specfun\besselmx.f`).

Sayfa 40–41 — 4.4 Diğer veri tipleri: single

`double` tek aritmetik tip değildir; IEEE tek hassasiyetine karşılık gelen **`single`** tipi de vardır (🔁 Java'daki `float`).

single neden kullanılır? 1. **Bellek tasarrufu:** single 32 bit, double 64 bit. 2000 elemanlı single vektör = 1000 elemanlı double vektör kadar yer. 2. **Doğruluk testi:** Algoritmayı hem single hem double ile çalıştırıp farkı, single çözümdeki hatanın tahmini olarak kullanmak.

Tablo 4.3 — single ve double:

Tip	Boyut	eps	Aralık
`single`	32 bit	$2^{-23} \approx 1.19 \times 10^{-7}$	10^±38
`double`	64 bit	$2^{-52} \approx 2.22 \times 10^{-16}$	10^±308

- Çoğu işlemcide single **kendi başına daha hızlı değildir**; ama az yer kapladığı için önbelleği (cache) daha iyi kullanıp dolaylı hız kazandırabilir.
- **Dezavantajlar:** yarı hassasiyet (~8 basamak, double'da 16) ve çok daha dar aralık (taşma/alttan taşma daha olası).

Örnek 4.22, 4.23, 4.24 — single'ın sınırları

```
>> eps('single')
ans =
   1.1921e-007
>> realmax('single')
ans =
   3.4028e+038
>> realmin('single')
ans =
   1.1755e-038
```

single oluşturmanın iki yolu: (1) `single` fonksiyonuyla dönüştürmek, (2) `eye`, `ones`, `zeros` fonksiyonlarına `'single'` girdisi eklemek (Bölüm 5.1).

Örnek 4.25 — pi'nin single ve double hali

```
>> format long, pi_s = single(pi), pi_d = pi
pi_s =
   3.1415927
pi_d =
   3.14159265358979
```

Örnek 4.26 — Fark

```
>> delta = pi_s - pi_d
delta =
   8.7422777e-008
```

Örnek 4.27 — single matris

```
>> A = ones(2,'single')
A =
     1     1
     1     1
```

Sayfa 42 — whos, class ve tamsayı tipleri

Örnek 4.28

```
>> whos
  Name       Size      Bytes  Class
  A          2x2          16  single array
  delta      1x1           4  single array
  pi_d       1x1           8  double array
  pi_s       1x1           4  single array

Grand total is 7 elements using 32 bytes
```

Bu örnekten üç sonuç: (1) single pi, double pi'den ~10^{-7} farklı (hassasiyetlerle uyumlu); (2) single, double'ın **yarısı kadar** yer kaplar; (3) `delta` (single − double) **single** çıktı.

⚠️ **Kural:** single ile double bir işlemde karşılaşırsa **sonuç single** olur. 🔁 Java'da `float + double` → `double` (genişe yükselir); MATLAB'da dar tipe iner.

- single aritmetiği MATLAB 7 ile geldi. Gömülü fonksiyonların çoğu (hepsi değil) single kabul eder; M-dosyası fonksiyonlarının çoğu double ister.

Örnek 4.29 ve 4.30 — `class` ile tip öğrenmek

```
>> class(pi)
ans =
double

>> class(single(pi))
ans =
single
```

MATLAB iki hassasiyeti tutarlı kullanır: single bir hesap taşarsa sonuçtaki `inf` / `-inf` da single olur.

Tamsayı tipleri (8 tane):

İşaretli	İşaretsiz	Bit
`int8`	`uint8`	8
`int16`	`uint16`	16
`int32`	`uint32`	32
`int64`	`uint64`	64

Başlıca kullanımları **görüntü verisini verimli saklamak**; sınırlı aritmetik desteklenir. `eye`, `ones`, `zeros`'a ek girdiyle oluşturulur.

Örnek 4.31

```
>> E = zeros(1,3,'int8')
E =
     0     0     0
```

Örnek 4.32

```
>> A = 5*ones(1,3,'uint16')
A =
     5     5     5
```

Örnek 4.33

```
>> whos
  Name      Size     Bytes  Class
  A         1x3          6  uint16 array
  E         1x3          3  int8 array

Grand total is 6 elements using 9 bytes
```

uint16: 3 × 2 byte = 6; int8: 3 × 1 byte = 3. (Aynı vektörler double olsaydı 24'er byte tutardı.)

Sayfa 43 — Tip dönüştürme ve sınırlar

Tipin adıyla aynı adlı fonksiyonlar o tipe dönüştürür; tipin aralığı `intmin` ve `intmax` ile bulunur.

Örnek 4.34 — Dönüştürürken **yuvarlar**

```
>> int8(30.8)
ans =
    31
```

🔁 Java'da `(int) 30.8` = 30 (keser); MATLAB 31 (yuvarlar).

Örnek 4.35

```
>> [intmin('int8') intmax('int8')]
ans =
  -128   127
```

Örnek 4.36 — Aralık dışı: uca sabitlenir

```
>> int8(128)
ans =
   127
```

🔁 Java'da `(byte) 128` = −128 (öbür uca sarar); MATLAB sınırda durur.

Örnek 4.37

```
>> [intmin('uint8') intmax('uint8')]
ans =
     0   255
```

Örnek 4.38

```
>> [uint8(-1) uint8(256)]
ans =
     0   255
```

Aralık dışındaki sayılar aralığın uçlarından birine eşlenir. Bütün veri tipleri ve ilişkileri: Şekil 18.1.

Bölüm sonu alıntıları: "*Yuvarlak sayılar her zaman yanlıştır.*" (Samuel Johnson) — "*Eksi çarpı eksi artıdır; nedenini tartışmamıza gerek yok.*" (W. H. Auden) — Cleve Moler: klasik MATLAB'daki düşük hassasiyeti taklit eden `chop` fonksiyonu ilginç bir merak konusuydu ama yuvarlama hatası analizinin yerini tutmazdı. Sayfa 44 boştur.

Bölüm 4 — Akılda kalması gerekenler

1. double: 8 byte, ~16 basamak, `eps` ≈ 2.2e-16, aralık ~10^±308.
2. Taşma → `Inf`; tanımsız → `NaN` (ve NaN bulaşıcıdır); alttan taşma → 0.
3. Aynı seviyedeki operatörler soldan sağa: `2^2^3` = 64. Şüphedeysen parantez.
4. Trigonometri radyanla; derece için `sind`, `cosd` …
5. single: 4 byte, ~8 basamak; double ile karışınca sonuç **single**.
6. Tamsayıya dönüşümde **yuvarlama**, aralık dışında **uca sabitlenme**.

Bölüm 5 — Matrisler

Alt başlıklar: **5.1 Matris oluşturma** (s. 45–50), **5.2 İndeksleme ve iki nokta gösterimi** (s. 50–53), **5.3 Matris ve dizi işlemleri** (s. 54–57), **5.4 Matris düzenleme** (s. 57–59), **5.5 Veri analizi** (s. 59–62).

Sayfa 45 — Giriş ve 5.1 Matris oluşturma

- **m×n matris:** m satır, n sütunluk iki boyutlu sayı dizisi. **Sütun vektörü:** n = 1. **Satır vektörü:** m = 1.
- MATLAB'ı lineer cebir için kullanmasan bile matris oluşturmayı ve düzenlemeyi bilmen gerekir.
- MATLAB'ın 3. ve önceki sürümlerinde tek veri tipi karmaşık matristi; bugün birçok tip var (Bölüm 18). *Dipnot şakası (Moler): "MATLAB güçlü tipli bir dildir: sadece bir veri tipi vardır!"*

`zeros` (sıfırlar), `ones` (birler), `eye` (birim matris: köşegen 1, gerisi 0). Üçünün yazımı aynı: `zeros(m,n)` veya `zeros([m,n])` → m×n; `zeros(n)` → n×n.

Örnek 5.1

```
>> zeros(2)
ans =
     0     0
     0     0
```

Örnek 5.2

```
>> ones(2,3)
ans =
     1     1     1
     1     1     1
```

Örnek 5.3 — Birim matris kare olmak zorunda değil

```
>> eye(3,2)
ans =
     1     0
     0     1
     0     0
```

- `eye(size(A))` → A ile aynı boyutta birim matris (`size`, Bölüm 3.2).
- `length(A)` → A'nın iki boyutundan **büyük olanı**. n×1 veya 1×n vektör için n'yi verir.

Sayfa 46 — Tablo 5.1, rand/randn, köşeli parantez

Tablo 5.1 — Temel matrisler:

Fonksiyon	Açıklama
`zeros`	Sıfırlar dizisi
`ones`	Birler dizisi
`eye`	Birim matris
`repmat`	Diziyi çoğaltıp döşer
`rand`	Düzgün dağılımlı rastgele sayılar
`randn`	Normal dağılımlı rastgele sayılar
`linspace`	Doğrusal aralıklı vektör
`logspace`	Logaritmik aralıklı vektör
`meshgrid`	3B grafikler için X ve Y dizileri
`:`	Düzenli aralıklı vektör ve matris indeksleme

- **`rand`**: [0,1] üzerinde **düzgün dağılım** (0 < a < b < 1 için [a,b] aralığına düşme oranı b − a). **`randn`**: **standart normal** (0,1) dağılım. Yazımları `eye` ile aynı; girdisiz çağrılırsa **tek** rastgele sayı üretirler.

Örnek 5.4

```
>> rand
ans =
    0.9528
```

Örnek 5.5

```
>> rand(3)
ans =
    0.7041    0.8407    0.5187
    0.9539    0.4428    0.0222
    0.5982    0.8368    0.3759
```

- Aynı rastgele sayıları tekrar üretmek için üretecin **durumu** ayarlanır: `rand('state',j)` — `j = 0` başlangıç durumu, `j` ≠ 0 → j'inci durum. `randn` da aynı şekilde. Periyotlar $2^{1492} \approx 10^{449}$'dan fazla. (Yeni sürümlerde `rng(j)`.)

Örnek 5.6 — Köşeli parantezle, satırlar alt alta (ilk 9 asal sayı)

```
>> A = [2  3  5
        7 11 13
       17 19 23]
A =
     2     3     5
     7    11    13
    17    19    23
```

Sayfa 47 — Yazım kuralları ve blok matrisler

Örnek 5.7 — Aynı matris tek satırda: satır sonu yerine `;`

```
>> A = [2 3 5; 7 11 13; 17 19 23]
```

Satır içinde elemanlar **boşluk** ya da **virgülle** ayrılır. ⚠️ Boşluk kullanıyorsan **+ / − işaretinden sonra boşluk bırakma**, yoksa toplama/çıkarma sanılır.

Örnek 5.8

```
>> v = [-1 2 -3 4]
v =
    -1     2    -3     4
```

Örnek 5.9

```
>> w = [-1, 2, -3, 4]
w =
    -1     2    -3     4
```

Örnek 5.10 — ⚠️ Tuzak

```
>> x = [-1 2 - 3 4]
x =
    -1    -1     4
```

`2 - 3` çıkarma oldu (−1); 4 yerine **3 eleman** çıktı.

Örnek 5.11 — Blok matris (`B = [1 2; 3 4]`)

```
>> C = [B        zeros(2)
        ones(2)  eye(2)]
C =
     1     2     0     0
     3     4     0     0
     1     1     1     0
     1     1     0     1
```

Örnek 5.12 — `blkdiag` : blok köşegen matris

```
>> A = blkdiag(2*eye(2),ones(2))
A =
     2     0     0     0
     0     2     0     0
     0     0     1     1
     0     0     1     1
```

Örnek 5.13 — `repmat(A,m,n)` : A'nın kopyalarından m×n blokluk matris (m verilmezse n)

```
>> A = repmat(eye(2),2)
A =
     1     0     1     0
     0     1     0     1
     1     0     1     0
     0     1     0     1
```

Sayfa 48 — Özel matrisler, toeplitz, hankel, gallery

Tablo 5.2 — Özel matrisler:

Fonksiyon	Matris
`compan`	Companion (eşlik) matrisi
`gallery`	Büyük test matrisi koleksiyonu
`hadamard`	Hadamard matrisi
`hankel`	Hankel matrisi
`hilb`	Hilbert matrisi
`invhilb`	Ters Hilbert matrisi
`magic`	Sihirli kare
`pascal`	Pascal matrisi
`rosser`	Klasik simetrik özdeğer test problemi
`toeplitz`	Toeplitz matrisi
`vander`	Vandermonde matrisi
`wilkinson`	Wilkinson özdeğer test matrisi

- **Hilbert matrisi:** (i,j) elemanı 1/(i+j−1); `hilb` ile, tersi (tamsayı elemanlı) `invhilb` ile üretilir.
- **`magic` :** sihirli kareler (satır, sütun, köşegen toplamları eşit).
- **Toeplitz:** her **köşegen** boyunca elemanlar sabit. **Hankel:** her **ters köşegen** boyunca sabit.

Örnek 5.14 — Toeplitz: **ilk sütun** ve **ilk satır** verilir

```
>> toeplitz([1 0 -1 -2],[1 2 4 8])
ans =
     1     2     4     8
     0     1     2     4
    -1     0     1     2
    -2    -1     0     1
```

Örnek 5.15 — Hankel: **ilk sütun** ve **son satır** verilir

```
>> hankel([3 1 2 0],[0 -1 -2 -3])
ans =
     3     1     2     0
     1     2     0    -1
     2     0    -1    -2
     0    -1    -2    -3
```

- **`gallery`**: N. J. Higham'ın test matrisi koleksiyonu (Tablo 5.3). Bilgi için `help private\matris_adı`. Bazıları **sparse** tipinde döner (Bölüm 15).

Örnek 5.16

```
>> help private\moler
 MOLER  Moler matrix (symmetric positive definite).
   A = GALLERY('MOLER',N,ALPHA) is the symmetric positive definite
   N-by-N matrix U'*U, where U = GALLERY('TRIW',N,ALPHA).
   For the default ALPHA = -1, A(i,j) = MIN(i,j)-2, and A(i,i) = i.
   One of the eigenvalues of A is small.
```

Sayfa 49 — gallery('moler') ve Tablo 5.3

Örnek 5.17

```
>> A = gallery('moler',5)
A =
     1    -1    -1    -1    -1
    -1     2     0     0     0
    -1     0     3     1     1
    -1     0     1     4     2
    -1     0     1     2     5
```

Kontrol: köşegen 1..5; A(1,2) = min(1,2)−2 = −1; A(3,4) = 3−2 = 1; A(4,5) = 4−2 = 2.

- Tablo 5.4, Tablo 5.2 ve 5.3'teki matrisleri özelliklerine göre sınıflandırır (özellikler çoğunlukla varsayılan argümanlar için geçerli; `randsvd` gibi bazılarında argümanlar uygun seçilmeli). Tanımlar Bölüm 9'da.
- Matris oluşturmanın bir başka yolu `load` ile dosyadan yüklemektir (s. 30).

Tablo 5.3 — `gallery` ile erişilebilen matrisler (s. 49–50):

Ad	Açıklama	Ad	Açıklama
`cauchy`	Cauchy matrisi	`lauchli`	Läuchli matrisi (dikdörtgen)
`chebspec`	Chebyshev spektral türev matrisi	`lehmer`	Lehmer matrisi (simetrik pozitif tanımlı)
`chebvand`	Chebyshev polinomları için Vandermonde benzeri	`leslie`	Leslie matrisi
`chow`	Chow matrisi (tekil Toeplitz alt Hessenberg)	`lesp`	Gerçek, hassas özdeğerli üç köşegenli
`circul`	Dairesel (circulant) matris	`lotkin`	Lotkin matrisi
`clement`	Clement matrisi (köşegeni sıfır, üç köşegenli)	`minij`	Simetrik pozitif tanımlı min(i,j)
`compar`	Karşılaştırma matrisleri	`moler`	Moler matrisi (simetrik pozitif tanımlı)
`condex`	Koşul sayısı tahmincileri için karşı örnekler	`neumann`	Ayrık Neumann probleminden tekil (sparse)
`cycol`	Sütunları döngüsel tekrar eden	`orthog`	Ortogonal ve neredeyse ortogonal
`dorr`	Dorr: köşegen baskın, kötü koşullu, üç köşegenli (sparse)	`parter`	Parter: tekil değerleri π yakını Toeplitz
`dramadah`	Tersi büyük tamsayılı 0-1 matrisi	`pei`	Pei matrisi
`fiedler`	Fiedler matrisi (simetrik)	`poisson`	Poisson denkleminden blok üç köşegenli (sparse)
`forsythe`	Forsythe: bozulmuş Jordan bloğu	`prolate`	Prolate: simetrik kötü koşullu Toeplitz
`frank`	Frank: kötü koşullu özdeğerler	`randcolu`	Normalleştirilmiş sütunlu, tekil değerleri verilen rastgele
`gearmat`	Gear matrisi	`randcorr`	Özdeğerleri verilen rastgele korelasyon
`grcar`	Grcar: hassas özdeğerli Toeplitz	`randhess`	Rastgele ortogonal üst Hessenberg
`hanowa`	Özdeğerleri dikey bir doğru üzerinde	`randjorth`	Rastgele J-ortogonal
`house`	Householder matrisi (3 çıktı)	`rando`	Elemanları −1, 0, 1 olan rastgele
`invhess`	Üst Hessenberg matrisin tersi	`randsvd`	Tekil değerleri ve bant genişliği verilen rastgele
`invol`	Involutory (kendinin tersi) matris	`redheff`	Redheffer'ın 0-1 matrisi
`ipjfact`	Faktöriyel elemanlı Hankel (2 çıktı)	`riemann`	Riemann hipoteziyle ilişkili
`jordbloc`	Jordan blok matrisi	`ris`	Ris: simetrik Hankel
`kahan`	Kahan: üst yamuk	`smoke`	Smoke: karmaşık, "duman halkası" sözde spektrumlu
`kms`	Kac–Murdock–Szegö Toeplitz	`toeppd`	Simetrik pozitif tanımlı Toeplitz
`krylov`	Krylov matrisi	`toeppen`	Beş köşegenli Toeplitz (sparse)
		`tridiag`	Üç köşegenli (sparse)
		`triw`	Wilkinson ve diğerlerinin üst üçgen matrisi
		`wathen`	Wathen: sonlu eleman matrisi (sparse, rastgele)
		`wilk`	Wilkinson'ın çeşitli matrisleri (2 çıktı)
		`gallery(3)`	Kötü koşullu 3×3
		`gallery(5)`	İlginç özdeğer problemi

Sayfa 50–51 — 5.2 İndeksleme ve iki nokta (`:`) gösterimi

- **`i:j`** → i'den j'ye 1'er artan satır vektörü. **`i:s:j`** → s adımla. i, j, s tamsayı olmak zorunda değil.

Örnek 5.18

```
>> 1:5
ans =
     1     2     3     4     5
```

Tablo 5.4 — Özelliklerine göre matrisler (çoğu `gallery` ile):

Özellik	Matrisler
Defective	chebspec, gallery(5), gearmat, jordbloc, triw
Hankel	hilb, ipjfact, ris
Hessenberg	chow, frank, grcar, randhess, randsvd
Idempotent	invol
Üç köşegenli matrisin tersi	kms, lehmer, minij
Involutory	invol, orthog, pascal
Nilpotent	chebspec, gallery(5)
Normal (simetrik/ortogonal değil)	circul
Ortogonal	hadamard, orthog, randhess, randsvd
Dikdörtgen	chebvand, cycol, kahan, krylov, lauchli, rando, randsvd, triw
Simetrik belirsiz	clement, fiedler
Simetrik pozitif tanımlı	hilb, invhilb, ipjfact, kms, lehmer, minij, moler, pascal, pei, poisson, prolate, randcorr, randsvd, toeppd, tridiag, wathen
Toeplitz	chow, dramadah, grcar, kms, parter, prolate, toeppd, toeppen
Tamamen pozitif/negatif olmayan	cauchy ($0 < x_1 < \ldots < x_n$ ve $0 < y_1 < \ldots < y_n$ ise), hilb, lehmer, pascal
Üç köşegenli	clement, dorr, lesp, randsvd, tridiag, wilk, wilkinson
Üçgen	dramadah, jordbloc, kahan, pascal, triw

Örnek 5.19

```
>> 4:-1:-2
ans =
     4     3     2     1     0    -1    -2
```

Örnek 5.20

```
>> 0:.75:3
ans =
         0    0.7500    1.5000    2.2500    3.0000
```

İndeksleme kuralları:

Yazım	Anlamı
`A(i,j)`	i. satır, j. sütun (i, j ≥ 1; **sıfır/negatif indeks yok**)
`A(p:q,r:s)`	p–q satırları ile r–s sütunlarının kesişimi
`A(:,j)`	j. sütunun tamamı
`A(i,:)`	i. satırın tamamı
`end`	o boyuttaki son indeks; `A(end,:)` son satır
`A([i j k],[p q])`	i, j, k satırları ile p, q sütunlarının kesişimi

Tek başına `:` = `1:end` ("hepsi"). 🔁 Java'daki `A[0][0]` = MATLAB'da `A(1,1)`.

Sayfa 52 — İndeksleme örnekleri (asal sayı matrisi A)

Örnek 5.21

```
>> A
A =
     2     3     5
     7    11    13
    17    19    23
```

Örnek 5.22

```
>> A(2,1)
ans =
     7
```

Örnek 5.23

```
>> A(2:3,2:3)
ans =
    11    13
    19    23
```

Örnek 5.24

```
>> A(:,1)
ans =
     2
     7
    17
```

Örnek 5.25

```
>> A(2,:)
ans =
     7    11    13
```

Örnek 5.26 — Son sütun, tersten

```
>> A(end:-1:1,end)
ans =
    23
    13
     5
```

Örnek 5.27

```
>> A([1 3],[2 3])
ans =
     3     5
    19    23
```

Örnek 5.28 — `A(:)`: bütün elemanlar **sütun sütun** tek sütunda

```
>> B = A(:)
B =
     2
     7
    17
     3
    11
    19
     5
    13
    23
```

⚠️ MATLAB matrisleri **sütun sırasıyla** (column-major) saklar.

Sayfa 53 — Atama, skaler genişletme, linspace, silme

Örnek 5.29 — `A(:)` atamanın solunda: şekli koruyarak doldurur

```
>> A = zeros(3); A(:) = primes(23); A = A'
A =
     2     3     5
     7    11    13
    17    19    23
```

`primes(23)` = 23'e kadar asallar. Sütun sütun yerleşir; asalları satırlara dizmek için transpoz `A'` gerekir.

Örnek 5.30 — Skaler genişletme (sağ taraf tek eleman olunca)

```
>> A = ones(3);
>> A(2:3,2:3) = 0
A =
     1     1     1
     1     0     0
     1     0     0
```

Örnek 5.31 — `linspace(a,b,n)` : a ile b arasında **n** eşit aralıklı nokta (n verilmezse 100)

```
>> linspace(-1,1,9)
ans =
  Columns 1 through 7
   -1.0000   -0.7500   -0.5000   -0.2500         0    0.2500    0.5000
  Columns 8 through 9
    0.7500    1.0000
```

İki nokta gösteriminde **adım**, `linspace`'te **nokta sayısı** verilir.

Örnek 5.32 — `[]` boş matris (0×0); satıra/sütuna atanınca **siler**

```
>> A(2,:) = []
A =
     2     3     5
    17    19    23
```

Aynı etki: `A = A([1 3],:)`. Boş matris argüman listelerinde yer tutucu olarak da kullanılır (Bölüm 5.5).

Sayfa 54 — 5.3 Matris ve dizi işlemleri

MATLAB	Matematik
Sağ bölme `a/b`	a ÷ b
Sol bölme `a\b`	b ÷ a

- Matrislerde işlemler **matris anlamında** (lineer cebir kuralları) ya da **dizi anlamında** (eleman eleman) yapılabilir (Tablo 5.5).
- **Toplama/çıkarma** iki anlamda aynıdır; boyutlar eşit olmalı.
- `A*B` matris çarpımı: A'nın sütun sayısı = B'nin satır sayısı olmalı.
- `A\B` → `A*X = B`'nin çözümü X; `A/B` → `X*B = A`'nın çözümü (Bölüm 9.2).

Örnek 5.33

```
>> A = [1 2; 3 4], B = ones(2)
A =
     1     2
     3     4
B =
     1     1
     1     1
```

Örnek 5.34

```
>> A+B
ans =
     2     3
     4     5
```

Örnek 5.35 — Matris çarpımı (satır × sütun)

```
>> A*B
ans =
     3     3
     7     7
```

Örnek 5.36 — Sol bölme

```
>> A\B
ans =
    -1    -1
     1     1
```

Kontrol: A·[−1; 1] = [1; 1] ✓.

Eleman eleman çarpma/bölme için operatörün önüne **nokta**: `C = A.*B` → C(i,j) = A(i,j)·B(i,j); `C = A./B` → A(i,j)/B(i,j); `C = A.\B` = `B./A`.

Sayfa 55 — Tablo 5.5 ve üs alma

Örnek 5.37

```
>> A.*B
ans =
     1     2
     3     4
```

Örnek 5.38

```
>> B./A
ans =
    1.0000    0.5000
    0.3333    0.2500
```

Tablo 5.5 — Temel matris ve dizi işlemleri:

İşlem	Matris anlamı	Dizi anlamı
Toplama	`+`	`+`
Çıkarma	`-`	`-`
Çarpma	`*`	`.*`
Sol bölme	`\`	`.\`
Sağ bölme	`/`	`./`
Üs alma	`^`	`.^`

📌 **Nokta varsa işlem eleman eleman yapılır.**

Örnek 5.39 — `A^2` = A*A, `A.^2` = her elemanın karesi

```
>> A^2, A.^2
ans =
     7    10
    15    22
ans =
     1     4
     9    16
```

`.^`'da üs de dizi olabilir (boyutlar uyuşmalı ya da taban skaler olmalı). `x = [1 2 3]; y = [2 3 4]; Z = [1 2; 3 4];`

Örnek 5.40

```
>> x.^y
ans =
     1     8    81
```

Örnek 5.41

```
>> 2.^x
ans =
     2     4     8
```

Örnek 5.42

```
>> 2.^Z
ans =
     2     4
     8    16
```

Sayfa 56 — Matris kuvveti, transpoz, çarpımlar

- Matris üssü her kuvvet için tanımlı: n < 0 tamsayı ise `A^n` = `inv(A)^n`; p tamsayı değilse özdeğerlerle hesaplanır (A köşegenleştirilemezse ya da özdeğer sistemi kötü koşulluysa sonuç hatalı olabilir).
- **`A'`** → eşlenik transpoz (A gerçekse sadece transpoz). **`A.'`** → eşleniksiz transpoz. Fonksiyon halleri: `ctranspose(A)`, `transpose(A)`.
- x, y sütun vektörü ise **`x'*y`** iç çarpımdır (= `dot(x,y)`). 3 elemanlı vektörlerin **vektörel çarpımı**: `cross`.

`x = [-1 0 1]'; y = [3 4 5]';` — ikisi de 3×1 (sondaki `'` sütuna çevirir).

Örnek 5.43

```
>> x'*y
ans =
     2
```

(1×3)·(3×1) = 1×1: −3 + 0 + 5 = 2. (`x*y` hata verir: 3×1 · 3×1; `x*y'` 3×3 dış çarpım.)

Örnek 5.44

```
>> dot(x,y)
ans =
     2
```

Örnek 5.45

```
>> cross(x,y)
ans =
    -4
     8
    -4
```

Örnek 5.46 — Kronecker çarpımı: m×n A ve p×q B → mp×nq, (i,j) bloğu a_ij·B

```
>> A = [1 10; -10 100]; B = [1 2 3; 4 5 6; 7 8 9];
>> kron(A,B)
ans =
     1     2     3    10    20    30
     4     5     6    40    50    60
     7     8     9    70    80    90
   -10   -20   -30   100   200   300
   -40   -50   -60   400   500   600
   -70   -80   -90   700   800   900
```

Örnek 5.47 — Matrise skaler eklenince skaler matrise genişletilir

```
>> [4 3; 2 1] + 4
ans =
     8     7
     6     5
```

Sayfa 57 — Skaler işlemler, matris fonksiyonları, 5.4

Örnek 5.48

```
>> A = [1 -1] - 6
A =
    -5    -7
```

⚠️ Genişletme gerekmeden anlamlı olan atamada genişletme yapılmaz: bundan sonra `A = 1` yazılırsa A **skaler 1** olur, `ones(1,2)` değil.

Örnek 5.49 — Skalerle çarpma/bölme eleman eleman

```
>> [3 4 5; 4 5 6]/12
ans =
    0.2500    0.3333    0.4167
    0.3333    0.4167    0.5000
```

Bölüm 4.3'teki fonksiyonların çoğu matrise **eleman eleman** uygulanır. Lineer cebir anlamındaki matris fonksiyonlarının adı **m** ile biter: `expm`, `funm`, `logm`, `sqrtm` (Bölüm 9.9). `A = [2 2; 0 2]` için:

Örnek 5.50

```
>> sqrt(A)
ans =
    1.4142    1.4142
         0    1.4142
```

Örnek 5.51

```
>> sqrtm(A)
ans =
    1.4142    0.7071
         0    1.4142
```

Örnek 5.52 — Kontrol: sqrtm(A)² = A

```
>> ans*ans
ans =
    2.0000    2.0000
         0    2.0000
```

5.4 Matris düzenleme

Örnek 5.53 — `reshape(A,m,n)`: elemanları A'dan **sütun sütun** alan m×n matris

```
>> A = [1 4 9; 16 25 36], B = reshape(A,3,2)
A =
     1     4     9
    16    25    36
B =
     1    25
    16     9
     4    36
```

A sütun sütun: 1, 16, 4, 25, 9, 36 → B'ye sütun sütun yerleşir.

Sayfa 58 — Tablo 5.6 ve `diag`

Tablo 5.6 — Matris düzenleme fonksiyonları:

Fonksiyon	İşlevi
`reshape`	Boyut değiştir
`diag`	Köşegen matrisler ve matrisin köşegenleri
`blkdiag`	Blok köşegen matris
`tril`	Alt üçgen kısmı al
`triu`	Üst üçgen kısmı al
`fliplr`	Sağ-sol çevir
`flipud`	Yukarı-aşağı çevir
`rot90`	90° döndür

`diag(x)` (x vektör) → ana köşegeni x olan köşegen matris. `diag(x,k)` → x'i **k'ıncı köşegene** koyar (k > 0 üst, k < 0 alt, k = 0 ana).

Örnek 5.54

```
>> diag([1 2 3])
ans =
     1     0     0
     0     2     0
     0     0     3
```

Örnek 5.55

```
>> diag([1 2], 1)
ans =
     0     1     0
     0     0     2
     0     0     0
```

Örnek 5.56

```
>> diag([3 4], -2)
ans =
     0     0     0     0
     0     0     0     0
     3     0     0     0
     0     4     0     0
```

A matrisse `diag(A)` → ana köşegeni **sütun vektörü** olarak verir; aynı köşegenli köşegen matris için `diag(diag(A))`. `diag(A,k)` → k'ıncı köşegen. (A = asal sayı matrisi.)

Sayfa 59 — Köşegenler, üçgenler ve 5.5 Veri analizi

Örnek 5.57

```
>> diag(A)
ans =
     2
    11
    23
```

Örnek 5.58

```
>> diag(A,-1)
ans =
     7
    19
```

`tril(A)` → alt üçgen (köşegen ve altı), `triu(A)` → üst üçgen (köşegen ve üstü). `tril(A,k)` / `triu(A,k)` → k'ıncı köşegenden itibaren.

Örnek 5.59

```
>> tril(A)
ans =
     2     0     0
     7    11     0
    17    19    23
```

Örnek 5.60

```
>> triu(A,1)
ans =
     0     3     5
     0     0    13
     0     0     0
```

Örnek 5.61

```
>> triu(A,-1)
ans =
     2     3     5
     7    11    13
     0    19    23
```

5.5 Veri analizi

Örnek 5.62

```
>> x = [4 -8 -2 1 0]
x =
     4    -8    -2     1     0
```

Örnek 5.63

```
>> [min(x) max(x)]
ans =
    -8     4
```

Örnek 5.64

```
>> sort(x)
ans =
    -8    -2     0     1     4
```

Sayfa 60 — Tablo 5.7, sort ayrıntıları, matrislerde sütun sütun

Örnek 5.65

```
>> sum(x)
ans =
    -5
```

Tablo 5.7 — Temel veri analizi fonksiyonları:

Fonksiyon	İşlevi
`max`	En büyük eleman
`min`	En küçük eleman
`mean`	Ortalama
`median`	Ortanca
`std`	Standart sapma
`var`	Varyans
`sort`	Artan sıralama
`sum`	Elemanların toplamı
`prod`	Elemanların çarpımı
`cumsum`	Kümülatif toplam
`cumprod`	Kümülatif çarpım
`diff`	Elemanların farkı

`sort` varsayılan olarak artan sıralar; azalan için `'descend'`. **Karmaşık vektörler mutlak değere göre** sıralanır.

Örnek 5.66

```
>> x = [1+i -3-4i 2i 1];
>> sort(x,'descend')
ans =
  -3.0000 - 4.0000i        0 + 2.0000i   1.0000 + 1.0000i   1.0000
```

Mutlak değerler: 5, 2, 1.414, 1.

- `sort` NaN'leri **en sona** koyar; `max` ve `min` NaN'leri **yok sayar**.
- Matrislerde bu fonksiyonlar **sütun sütun** çalışır: `max` / `min` her sütunun en büyüğünü/küçüğünü, `sum` sütun toplamlarını verir, `sort` her sütunu sıralar. `min` / `max` ikinci çıktı olarak **konumu** verebilir.

A = [0 −1 2; 1 2 −4; 5 −3 −4] için:

Örnek 5.67

```
>> max(A)
ans =
     5     2     2
```

Sayfa 61 — min/max ayrıntıları ve diff

Örnek 5.68

```
>> [m,i] = min(A)
m =
     0    -3    -4
i =
     1     3     2
```

Bir sütunda birden fazla en küçük varsa **ilkinin** indeksi verilir (3. sütunda −4 hem 2. hem 3. satırda).

Örnek 5.69 — Matrisin en küçük elemanı: `min` iki kez

```
>> min(min(A))
ans =
    -4
```

Örnek 5.70 — Alternatif (2'den fazla boyutlu dizilerde de çalışır)

```
>> min(A(:))
ans =
    -4
```

Örnek 5.71 — Satır satır: üçüncü argüman ile 2. boyut

```
>> max(A,[],2)
ans =
     2
     2
     5
```

`[]` yer tutucu: iki argümanla `max` başka bir şey yapar.

Örnek 5.72 — İki argümanlı `max`: eleman eleman en büyük

```
>> max(A,0)
ans =
     0     0     2
     1     2     0
     5     0     0
```

- `sort` ve `sum` da ikinci argümanla satır satır çalışır (örn. `sum(A,2)`); `sort` için Bölüm 21.3.
- Karmaşık verilerde `max` / `min` mutlak değere göre karşılaştırır.
- **`diff`**: n elemanlı x için `[x(2)-x(1) x(3)-x(2) ... x(n)-x(n-1)]` (n−1 eleman).

Örnek 5.73

```
>> x = (1:8).^2
x =
     1     4     9    16    25    36    49    64
```

Örnek 5.74

```
>> y = diff(x)
y =
     3     5     7     9    11    13    15
```

Sayfa 62 — Eksik veri (NaN)

Örnek 5.75

```
>> z = diff(y)
z =
     2     2     2     2     2     2
```

Veri analizinde **NaN genellikle eksik veriyi** temsil eder (ölçülemeyen değer; örnek Bölüm 22.4). NaN içeren her hesap NaN verdiği için önce NaN'ler **temizlenmelidir**; yollar `isnan` kullanır (Bölüm 6.1).

Örnek 5.76

```
>> x = [2 1 NaN -1 6], y = x;
x =
     2     1   NaN    -1     6
```

Örnek 5.77

```
>> mean(x)
ans =
   NaN
```

Örnek 5.78 — Yöntem 1: NaN **olmayanları seç**

```
>> x = x(~isnan(x)), mean(x)
x =
     2     1    -1     6
ans =
     2
```

`isnan(x)` = [0 0 1 0 0]; `~isnan(x)` = [1 1 0 1 1]; `x(...)` 1 olan yerleri alır. (2+1−1+6)/4 = 2.

Örnek 5.79 — Yöntem 2: NaN **olanları sil**

```
>> y(isnan(y)) = [], mean(y)
y =
     2     1    -1     6
ans =
     2
```

Bölüm sonu alıntıları: Kahan — doğru kullanılan boş diziler, alt matrislerden özyinelemeli matris kuran algoritmalarda baştaki ve sondaki özel durumlarla uğraşma zahmetini ortadan kaldırır. *Star Trek III* — "Matris bir günde oluştu." Halmos — "2×2'lik bir matrise bakarak başlarım... Genelde 2×2 ya da 3×3 yeterlidir."

Bölüm 5 — Akılda kalması gerekenler

1. `zeros`, `ones`, `eye`, `rand`: `(n)` kare, `(m,n)` dikdörtgen.
2. `;` yeni satır, boşluk/`,` yeni eleman. `[1 - 2]` = çıkarma!
3. İndeksler 1'den başlar. `:` = hepsi, `end` = son.
4. `A(:)` sütun sütun tek sütun; `[]` atamak siler.
5. **Nokta = eleman eleman:** `.*`, `./`, `.^`. `A'` transpoz, `A\B` → `A*X = B` çözümü.
6. Veri analizi fonksiyonları matriste **sütun sütun**; satır satır için `max(A,[],2)`.
7. NaN temizleme: `x(~isnan(x))` ya da `x(isnan(x)) = []`.

Bölüm 6 — Operatörler ve Akış Kontrolü

Alt başlıklar: **6.1 İlişkisel ve mantıksal operatörler** (s. 63–69), **6.2 Akış kontrolü** (s. 69–73).

Sayfa 63 — 6.1 Mantıksal veri tipi ve ilişkisel operatörler

MATLAB'da **mantıksal (logical)** veri tipi vardır: **1 = doğru**, **0 = yanlış** (🔁 Java'daki `boolean`).
İlişkisel/mantıksal operatörler ve `true`, `false` fonksiyonlarıyla üretilir.

Örnek 6.1

```
>> a = true
a =
     1
```

Örnek 6.2

```
>> b = false
b =
     0
```

Örnek 6.3

```
>> c = 1
c =
     1
```

Örnek 6.4

```
>> whos
  Name      Size     Bytes  Class
  a         1x1          1  logical array
  b         1x1          1  logical array
  c         1x1          8  double array

Grand total is 3 elements using 10 bytes
```

Mantıksal değerler **1 byte**, double **8 byte**. Ekranda ikisi de 1 görünse de tipleri farklı.

İlişkisel operatörler:

MATLAB	Anlamı	Java
`==`	eşit	`==`
`~=`	eşit değil	`!=` ⚠️
`<`	küçük	`<`
`>`	büyük	`>`
`<=`	küçük veya eşit	`<=`
`>=`	büyük veya eşit	`>=`

Tek `=` **her zaman atamadır**, eşitlik testi değildir. Skalerlerin karşılaştırması 1 ya da 0 verir.

Sayfa 64 — Matris karşılaştırmaları, isequal, isinf, isreal

Aynı boyuttaki iki matris **eleman eleman**, matris ile skaler ise skaler her elemanla karşılaştırılır; sonuç 0/1 matrisidir. `A = [1 2; 3 4]; B = 2*ones(2);`

Örnek 6.5

```
>> A == B
ans =
     0     1
     0     0
```

Örnek 6.6

```
>> A > 2
ans =
     0     0
     1     1
```

Örnek 6.7 — İki dizinin **tamamen** eşit olup olmadığı (aynı boyut, aynı elemanlar)

```
>> isequal(A,B)
ans =
     0
```

`isequal`, adı **is** ile başlayan çok sayıda mantıksal fonksiyondan biridir (Tablo 6.1; tam liste `doc is`).

Örnek 6.8 — `isinf(A)`: ±inf olan yerlerde 1

```
>> A = [1 inf; -inf NaN];
>> isinf(A)
ans =
     0     1
     1     0
```

⚠️ **`isnan` özellikle önemlidir**, çünkü **`x == NaN` her zaman 0 (yanlış) verir**, x NaN olsa bile! (NaN hiçbir şeyle, kendisiyle bile, eşit değildir.)

`isreal`: Bir dizi matematikte gerçek olduğu halde `isreal` ona gerçek demeyebilir. `isreal(A)`, A'nın **hiç sanal kısmı yoksa** doğrudur; dizinin nasıl oluşturulduğu belirleyicidir.

Örnek 6.9

```
>> a = 1;
>> b = complex(1,0);
>> c = 1 + 0i;
>> [a b c]
ans =
     1     1     1
```

`complex(a,b)` = a + bi. `complex(1,0)` sanal kısım sıfır olsa bile **saklar**; `1 + 0i` ise sıfır sanal kısmı atar.

Sayfa 65 — Tablo 6.1, whos, isreal, mantıksal operatörler

Tablo 6.1 — Seçilmiş `is*` fonksiyonları:

Fonksiyon	Ne test eder
`ischar`	Karakter dizisi (string) mi
`isempty`	Boş dizi mi
`isequal`	Diziler eşit mi
`isequalwithequalnans`	Eşit mi (NaN'ler eşit sayılır)
`isfinite`	Sonlu elemanlar
`isfloat`	Ondalıklı dizi mi (single veya double)
`isinf`	Sonsuz elemanlar
`isinteger`	Tamsayı dizisi mi
`islogical`	Mantıksal dizi mi
`isnan`	NaN elemanlar
`isnumeric`	Sayısal dizi mi (tamsayı veya ondalıklı)
`isreal`	Gerçek dizi mi
`isscalar`	Skaler mi (1×1)
`issorted`	Sıralı vektör mü
`isvector`	Vektör mü (1×n veya n×1)

Örnek 6.10

```
>> whos a b c
  Name      Size     Bytes  Class
  a         1x1          8  double array
  b         1x1         16  double array (complex)
  c         1x1          8  double array

Grand total is 3 elements using 32 bytes
```

Örnek 6.11

```
>> [isreal(a), isreal(b), isreal(c)]
ans =
     1     0     1
```

Mantıksal operatörler:

MATLAB	Anlamı	Java
`&`	mantıksal VE	`&`
`&&`	VE (sadece skaler, kısa devreli)	`&&`
`|`	mantıksal VEYA	`|`
`||`	VEYA (sadece skaler, kısa devreli)	`||`
`~`	mantıksal DEĞİL	`!` ⚠️
`xor`	dışlayıcı VEYA	`^`
`all`	vektörün bütün elemanları sıfırdan farklıysa doğru	—
`any`	herhangi bir elemanı sıfırdan farklıysa doğru	—

`&`, `|`, `~` matrisle 0/1 matrisi üretir. `x = [-1 1 1]; y = [1 2 -3];` → `x>0` = [0 1 1], `y>0` = [1 1 0].

Sayfa 66 — Mantıksal örnekler, && ve kısa devre

Örnek 6.12

```
>> x>0 & y>0
ans =
     0     1     0
```

Örnek 6.13

```
>> x>0 | y>0
ans =
     1     1     1
```

Örnek 6.14

```
>> xor(x>0,y>0)
ans =
     1     0     1
```

Örnek 6.15

```
>> any(x>0)
ans =
     1
```

Örnek 6.16

```
>> all(x>0)
ans =
     0
```

- `xor` fonksiyon olarak çağrılır: `xor(a,b)`. VE, VEYA, DEĞİL ve ilişkisel operatörlerin fonksiyon halleri de vardır: `and(a,b)`, `eq(a,b)` … (`help ops`).
- **`&&` ve `||`**: (1) **sadece skalerle** çalışır; skaler ifadelerde `&` ve `|` yerine tercih edilmeli.

Örnek 6.17

```
>> any(x>0) && any(y>0)
ans =
     1
```

Örnek 6.18 — Vektörle hata

```
>> x>0 && y>0
??? Operands to the || and && operators must be convertible to
    logical scalar values.
```

(2) **Kısa devre:** `ifade1 && ifade2`'de ifade1 yanlışsa ifade2 **hesaplanmaz**; `ifade1 || ifade2`'de ifade1 doğruysa ifade2 hesaplanmaz. Hesaptan tasarruf eder, uyarı ve hataları önler.

Örnek 6.19 — Sıfıra bölmeyi önlemek

```
if x > 0 && sin(1/x) < 0.5
```

Öncelik Tablo 6.2'de (`help precedence`). Aynı seviyede soldan sağa. **VE, VEYA'dan önce gelir**: `x | y & z` = `x | (y & z)`.

Sayfa 67 — Tablo 6.2, all/any matriste, find

Tablo 6.2 — Operatör önceliği:

Seviye	Operatör
1 (en yüksek)	Parantez `()`
2	Transpoz `.'`, üs `.^`, eşlenik transpoz `'`, matris üssü `^`
3	Tekli artı `+`, tekli eksi `-`, mantıksal değil `~`
4	`.*`, `./`, `.\`, `*`, `/`, `\`
5	Toplama `+`, çıkarma `-`
6	İki nokta `:`
7	`<`, `<=`, `>`, `>=`, `==`, `~=`
8	Mantıksal VE `&`
9	Mantıksal VEYA `|`
10	Kısa devreli VE `&&`
11 (en düşük)	Kısa devreli VEYA `||`

Niyeti netleştirmek için parantez kullan: `x | (y & z)`.

- Matrislerde `all` her **sütuna** uygulanır ve satır vektörü verir. Bu yüzden `all(all(A==B))` A ile B'nin eşitliğini test eder; `any(any(A==B))` en az bir eşit eleman varsa 1. Alternatifler: `all(A(:)==B(:))`, `any(A(:)==B(:))`.
- `find` → **sıfırdan farklı** elemanların **indekslerini** döndürür.

Örnek 6.20

```
>> x = [-3 1 0 -inf 0];
>> f = find(x)
f =
     1     2     4
```

Örnek 6.21

```
>> x(f)
ans =
    -3     1  -Inf
```

Örnek 6.22 — Sadece sonlu elemanlar

```
>> x(find(isfinite(x)))
ans =
    -3     1     0     0
```

Sayfa 68 — find ile değiştirme, matriste find, mantıksal indeksleme

Örnek 6.23 — Negatifleri sıfırla

```
>> x(find(x < 0)) = 0
x =
     0     1     0     0     0
```

Matriste `find`'ın indeksleri `A(:)`**'ya göre** (sütunlar alt alta) verilir ve A'yı indekslemekte kullanılabilir.

Örnek 6.24

```
>> A = [4 2 16; 12 4 3], B = [12 3 1; 10 -1 7]
A =
     4     2    16
    12     4     3
B =
    12     3     1
    10    -1     7
```

Örnek 6.25

```
>> f = find(A<B)
f =
     1
     3
     6
```

Sütun sırasıyla: (1,1) 4<12 ✓, (2,1) ✗, (1,2) 2<3 ✓, (2,2) ✗, (1,3) ✗, (2,3) 3<7 ✓.

Örnek 6.26

```
>> A(f) = 0
A =
     0     0    16
    12     4     0
```

`[i,j] = find(A)` → sıfırdan farklı elemanların **satır ve sütun** indeksleri.

Mantıksal ve sayısal indeksleme: Mantıksal operatörlerin sonucu 0/1'lerden oluşan **mantıksal dizilerdir**; `logical` fonksiyonuyla da oluşturulur (0/1 dışındaki sıfırdan farklı değerler 1 olur, uyarı verilir). İkisi de indekslemede kullanılır ama **farklı çalışır**: - **Mantıksal dizi** → indeksin **doğru (1)**

olduğu yerlerdeki elemanları seçer. - **Sayısal dizi** → içindeki **sayıların gösterdiği konumlardaki** elemanları seçer.

Örnek 6.27

```
>> clear
>> y = [1 2 0 -3 0]
y =
     1     2     0    -3     0
```

Örnek 6.28 — Mantıksal maske

```
>> i1 = (y ~= 0)
i1 =
     1     1     0     1     0
```

Örnek 6.29 — Aynı sayılar, elle yazılmış (double)

```
>> i2 = [1 1 0 1 0]
i2 =
     1     1     0     1     0
```

Sayfa 69 — Mantıksal indeksleme sonuçları ve 6.2 Akış kontrolü

Örnek 6.30 — Mantıksal dizi ile: 1 olan yerler

```
>> y(i1)
ans =
     1     2    -3
```

Örnek 6.31 — Sayısal dizi ile: 0. eleman yok → hata

```
>> y(i2)
??? Subscript indices must either be real positive integers or
    logicals.
```

Örnek 6.32

```
>> whos i1 i2
  Name      Size     Bytes  Class
  i1        1x5          5  logical array
  i2        1x5         40  double array
```

Örnek 6.33

```
>> isequal(i1,i2)
ans =
     1
```

Değerleri eşit, ama indekslemede yalnızca `i1` çalışır.

Örnek 6.34 — Sayısal indeksle aynı sonuç: konum numaraları

```
>> i3 = [1 2 4]; y(i3)
ans =
     1     2    -3
```

- `find`'ın girdisi mantıksal diziyse `find` çoğu zaman gereksizdir: `x(find(isfinite(x)))` yerine `x(isfinite(x))`.
- Bölüm 5'teki `x(~isnan(x))` tam olarak bu **mantıksal indekslemedir**.

Örnek 6.35 — Mantıksal değerlerle toplama/çarpma: sonuç double

```
>> a = true; b = false; c = 2*a+b, class(c)
c =
     2
ans =
double
```

Örnek 6.36 — Diğer işlemler başarısız olur

```
>> b/a
??? Function 'mrdivide' is not defined for values of class 'logical'.

Error in ==> mrdivide at 16
  builtin('mrdivide', varargin{:});
```

6.2 Akış kontrolü

Dört yapı: **`if`**, **`for`**, **`while`**, **`switch`**. En basit `if`:

```
if ifade
    komutlar
end
```

Komutlar, `ifade`nin **bütün elemanları sıfırdan farklıysa** çalışır. 🔁 Parantez zorunlu değil; `{ }` yerine **`end`**.

Sayfa 70 — if, else, elseif, for

Örnek 6.37 — x > y ise yer değiştir

```
if x > y
   temp = y;
   y = x;
   x = temp;
end
```

Örnek 6.38 — Tek satır: `if`'ten sonra **virgül** gerekir

```
if x > 0, x = sqrt(x); end
```

Örnek 6.39 — `else`: koşul yanlışsa

```
e = exp(1);
if 2^e > e^2
   disp('2^e is bigger')
else
   disp('e^2 is bigger')
end
```

2^e ≈ 6.58, e² ≈ 7.39 → ekrana `e^2 is bigger` yazılır.

Örnek 6.40 — `elseif` (⚠️ bitişik yazılır, `else if` değil)

```
if isnan(x)
   disp('Not a Number')
elseif isinf(x)
   disp('Plus or minus infinity')
else
   disp('A ''regular'' floating point number')
end
```

Üçüncü `disp`'teki `''` ekrana **tek tırnak** (`'`) yazdırır.

`for` döngüsü: En kullanışlı yapılardan biri; ama hızlı ve kısa kod için deneyimli programcılar mümkünse `for`'dan kaçınır (vektörleştirme, Bölüm 20.1).

```
for değişken = ifade
    komutlar
end
```

`ifade` genelde `i:s:j` vektörüdür; değişken sırayla her elemanı alır.

Örnek 6.41 — Harmonik serinin ilk 25 terimi

```
>> s = 0;
>> for i = 1:25, s = s + 1/i; end, s
s =
    3.8160
```

Sayfa 71 — for (devam), while, break

Örnek 6.42 — `ifade` köşeli parantezle

```
>> for x = [pi/6 pi/4 pi/3], disp([x, sin(x)]), end
    0.5236    0.5000
    0.7854    0.7071
    1.0472    0.8660
```

🔁 Java'daki for-each: `for (double x : dizi)`.

Örnek 6.43 — İç içe döngüler (girinti okunabilirliği artırır): j ≥ i için (i,j) elemanı i/j olan 5×5 simetrik matris

```
n = 5; A = eye(n);
for j = 2:n
    for i = 1:j-1
        A(i,j) = i/j;
        A(j,i) = i/j;
    end
end
```

Sonuç (kitapta gösterilmez):

```
    1.0000    0.5000    0.3333    0.2500    0.2000
    0.5000    1.0000    0.6667    0.5000    0.4000
    0.3333    0.6667    1.0000    0.7500    0.6000
    0.2500    0.5000    0.7500    1.0000    0.8000
    0.2000    0.4000    0.6000    0.8000    1.0000
```

- `ifade` matris olabilir: değişken sırayla **sütunları** alır. Örn. `for x = eye(n), ..., end` → x sırayla birim vektörler.

`while`:

```
while ifade
    komutlar
end
```

`ifade` doğru olduğu sürece tekrar eder.

Örnek 6.44 — En küçük pozitif ondalıklı sayı

```
>> x = 1; while x>0, xmin = x; x = x/2; end, xmin
xmin =
   4.9407e-324
```

(Bölüm 4'teki `realmin*eps` ile aynı.)

- **`break`** → döngüyü bitirir, `end`'den sonraki ilk komuta geçer. **`while 1, ..., end`** sonsuz döngü kurar (çıkış testini başa koymak uygun değilse). MATLAB'da **repeat-until** (🔁 Java `do-while`) yoktur.

Örnek 6.45 — Aynı örnek `break` ile

```
x = 1;
while 1
   xmin = x;
   x = x/2;
   if x == 0, break, end
end
xmin
```

- `break` `for` döngüsünden de çıkar. İç içe döngülerde **sadece bir üst seviyeye** çıkar.
- **`continue`** → döngünün kalanını atlayıp **bir sonraki tura** geçer.

Sayfa 72 — continue ve switch

Örnek 6.46

```
for i=1:10
    if i < 5, continue, end
    disp(i)
end
```

Ekrana **5'ten 10'a** kadar sayılar yazılır. Karmaşık döngülerde `continue` uzun `if` bloklarından kurtarır.

`switch`: `switch ifade`, ardından `case ifade komutlar` listesi, isteğe bağlı `otherwise komutlar`, sonra `end`. Eşleşen **ilk** `case` çalışır; hiçbiri eşleşmezse `otherwise`.

Örnek 6.47 — p-norm (`norm(x,p)`), sadece üç p için

```
switch p
    case 1
        y = sum(abs(x));
    case 2
        y = sqrt(x'*x);
    case inf
        y = max(abs(x));
    otherwise
        error('p must be 1, 2 or inf.')
end
```

`error` için Bölüm 14.1.

Örnek 6.48 — `case` içinde değer listesi (süslü parantez = hücre dizisi, Bölüm 18.3)

```
x = input('Enter a real number: ');
switch x
    case {inf,-inf}
        disp('Plus or minus infinity')
    case 0
        disp('Zero')
    otherwise
        disp('Nonzero and finite')
end
```

⚠️ **C/Java'dan farkı:** Eşleşen `case` çalıştıktan sonra kontrol `switch`'in sonrasına geçer; **`break` gerekmez** (fall-through yok). `default` yerine `otherwise`.

Sayfa 73 — Alıntılar

Star Trek IV — Kirk: "Bu küçük sorunu nasıl çözmeyi öneriyorsun?" Spock: "Basit mantık yeterli olacaktır." — Kaufman (*A FORTRAN Coloring Book*): "Öncelik sırası aynı olanlar soldan sağa hesaplanır. Şüphedeysen birkaç parantez ekle." Sayfa 74 boştur.

Bölüm 6 — Akılda kalması gerekenler

1. Eşit değil `~=`, değil `~` (Java: `!=`, `!`).
2. Karşılaştırma matrislerde eleman eleman, sonuç 0/1 matrisi.
3. `x == NaN` her zaman yanlış → `isnan(x)` kullan.
4. `&&` / `||` sadece skalerle ve kısa devreli; vektörlerde `&` / `|`.
5. `find` → konumlar; mantıksal dizi ile indeksleme → 1 olan yerler.
6. Bloklar `end` ile biter; `elseif` bitişik; `do-while` yok; `switch`'te `break` gerekmez, `default` yerine `otherwise`.

Bölüm 7 — M-Dosyaları (M-Files)

Alt başlıklar: **7.1 Script'ler ve fonksiyonlar** (s. 75–81), **7.2 İsim verme ve düzenleme** (s. 81–82), **7.3 M-dosyaları ve MATLAB yolu** (s. 82–83), **7.4 Başlangıç** (s. 83–84), **7.5 Komut/fonksiyon ikiliği** (s. 84–85).

Sayfa 75 — 7.1 Script'ler ve fonksiyonlar

M-dosyaları, diğer dillerdeki **programların, fonksiyonların, alt programların ve prosedürlerin** karşılığıdır. Komutları bir M-dosyasında toplamanın faydaları: - Uzun komut listesini tekrar yazmak yerine dosyayı düzenleyerek algoritma denemek, - Sayısal bir deneyin kalıcı kaydını tutmak, - Sonra tekrar kullanılacak araçlar oluşturmak, - M-dosyalarını başkalarıyla paylaşmak (internetteki M-dosyaları: Ek C).

M-dosyası = MATLAB komutları içeren `.m` **uzantılı metin dosyası**. İki türü vardır:

	Script M-dosyası (komut dosyası)	Fonksiyon M-dosyası
İlk satır	Herhangi bir komut	`function` tanım satırı
Girdi/çıktı	Yok	Girdi alır, çıktı döndürür
Değişkenler	Çalışma alanındaki (workspace) değişkenler	Fonksiyona **yerel** (`global` hariç)
🔁 Java	`main` içindeki kod parçası	Metot

Basit bir script örneği `marks.m` idi (Bölüm 2.1). İkinci örnek **"özdeğer ruleti"**: rastgele gerçek 8×8 matrisin kaç **gerçek özdeğeri** (eigenvalue; `eig(A)`, Bölüm 9) olduğu sayılır. Cevap **0, 2, 4, 6 veya 8**'dir (her zaman çift), çünkü gerçek matrisin gerçek olmayan özdeğerleri **eşlenik çiftler** halinde gelir.

Örnek 7.1 — `spin.m` script'i

```
%SPIN
% Counts number of real eigenvalues of random matrix.
A = randn(8); sum(abs(imag(eig(A)))<.0001)
```

İçten dışa: `eig(A)` özdeğerler → `imag` sanal kısımlar → `abs` mutlak değer → `< .0001` sanal kısmı (biraz keyfi seçilmiş) 10^{-4} eşiğinden küçük olanlar 1 → `sum` sayar. İlk iki satır `%` ile başladığı için **yorumdur**; MATLAB `%`'den sonrasını yok sayar.

Sayfa 76 — spin çağrıları, rouldist, fonksiyonlar

`spin` yazmak iki komutu yazmakla aynıdır ve "rulet çarkını çevirir". Her çağrı farklı rastgele matris üretir:

Örnek 7.2

```
>> spin
ans =
     2
```

Örnek 7.3

```
>> spin
ans =
     4
```

Örnek 7.4 — Listing 7.1: `rouldist.m` (1000 matris, histogram)

```
%ROULDIST    Empirical distribution of number of real eigenvalues.

k = 1000;
wheel = zeros(k,1);
for i = 1:k
    A = randn(8);
    % Count number of eigenvalues with imag. part < tolerance.
    wheel(i) = sum(abs(imag(eig(A)))<.0001);
end
hist(wheel,[0 2 4 6 8]);
```

`wheel` sonuçları saklar; `hist` sonuçların histogramını çizer (Şekil 7.1). Okunabilirlik için `for` içi girintili ve ilk komuttan önce boş satır bırakılmış. Kesin olasılıklar kaynakça [23], [24]'te.

Fonksiyon M-dosyaları girdi alıp çıktı döndüren **kendi fonksiyonlarını** yazmanı sağlar; `sin`, `eye`, `size` gibi aynen kullanılır.

- **İlk satır:** `function` anahtar kelimesi, çıktı argümanı, `=`, fonksiyon adı, parantez içinde girdi argümanı. Genel olarak istenen sayıda girdi ve çıktı olabilir.
- ⚠️ **Fonksiyon adı = `.m` dosyasının adı** (örn. `maxentry.m`).
- **İkinci satır = H1 (help 1) satırı:** `%` + boşluksuz **BÜYÜK harfle** fonksiyon adı + bir veya daha fazla boşluk + kısa açıklama (büyük harfle başlar, **nokta** ile biter, "the" ve "a" kullanılmaz).

Sayfa 77 — Şekil 7.1, help ve fonksiyon çağrıları

Şekil 7.1: `rouldist` histogramı (1000 denemede yaklaşık: 0 → ~70, **2 → ~580**, 4 → ~320, 6 → ~20, 8 → ~0).

- İlk yorum satırından ilk yorum olmayan satıra (genelde boş satır) kadar bütün yorumlar **`help fonksiyon_adı`** ile gösterilir; bu satırlar fonksiyonu ve argümanlarını açıklamalı, adlar BÜYÜK harfle yazılmalıdır.

Örnek 7.5 — Listing 7.2: `maxentry.m` (mutlak değerce en büyük eleman; sayfa 78'de)

```
function y = maxentry(A)
%MAXENTRY   Largest absolute value of matrix entries.
%           MAXENTRY(A) is the maximum of the absolute values
%           of the entries of A.

y = max(max(abs(A)));
```

🔁 `return y;` yazılmaz; çıktı değişkenine (`y`) atama yeterli.

Örnek 7.6

```
>> help maxentry
 MAXENTRY   Largest absolute value of matrix entries.
            MAXENTRY(A) is the maximum of the absolute values
            of the entries of A.
```

Kitabın tavsiyesi: **bütün** fonksiyonları böyle belgele; ilk yazılma tarihini ve değişiklikleri de yorumlara not et. `help` script'lerde de baştaki yorumları gösterir.

Örnek 7.7

```
>> maxentry(1:10)
ans =
    10
```

Örnek 7.8

```
>> mx = maxentry(magic(4))
mx =
    16
```

Sayfa 78 — Listing 7.2, 7.3 ve flogist

Örnek 7.9 — Listing 7.3: `flogist.m` — lojistik fonksiyon f(x) = x(1 − ax) ve türevi

```
function [f,fprime] = flogist(x,a)
%FLOGIST   Logistic function and its derivative.
%          [F,FPRIME] = FLOGIST(X,A) evaluates the logistic
%          function F(X) = X.*(1-A*X) and its derivative FPRIME
%          at the matrix argument X, where A is a scalar parameter.

f = x.*(1-a*x);
fprime = 1-2*a*x;
```

İki çıktı **köşeli parantez** içinde. Çağırırken bütün çıktıları istemek gerekmez, ama argümanlar **listenin sonundan başlayarak** bırakılır; birden fazla çıktı köşeli parantezle istenir.

Örnek 7.10

```
>> f = flogist(2,.1)
f =
    1.6000
```

2·(1 − 0.2) = 1.6.

Örnek 7.11

```
>> [f,fprime] = flogist(2,.1)
f =
    1.6000
fprime =
    0.6000
```

1 − 2·0.1·2 = 0.6.

Örnek 7.12 — `.*` sayesinde vektörle aynı anda her elemana

```
>> flogist(1:4,2)
ans =
    -1    -6   -15   -28
```

x(1 − 2x): 1·(−1), 2·(−3), 3·(−5), 4·(−7).

Sayfa 79 — cheby ve sqrtn

Örnek 7.13 — Listing 7.4: `cheby.m` — Chebyshev polinomları (Şekil 8.11'de `fplot` ile kullanılır)
$T_0(x) = 1$, $T_1(x) = x$, $T_k(x) = 2x \cdot T_{k-1}(x) - T_{k-2}(x)$ $(k \geq 2)$.

```
function Y = cheby(x,p)
%CHEBY    Chebyshev polynomials.
%         Y = CHEBY(X,P) evaluates the first P Chebyshev polynomials
%         at the vector X. The K'th column of Y contains the
%         Chebyshev polynomial of degree K-1 evaluated at X.

Y = ones(length(x),p);
x = x(:);     % Ensure x is a column vector.
if p == 1, return, end

Y(:,2) = x;
for k = 3:p
    Y(:,k) = 2*x.*Y(:,k-1) - Y(:,k-2);
end
```

Y'nin **i'inci satırı** x(i) noktasında $T_0 \ldots T_{p-1}$ değerleri (`fplot` 'un istediği biçim). **`return`** M-dosyasından hemen çıkar; bazı dillerin aksine fonksiyonun sonuna `return` koymak gerekmez. (Kendi denemen için: `cheby([0 0.5 1],3)` → satırlar [1 0 −1], [1 0.5 −0.5], [1 1 1].)

Newton yöntemiyle √a (Heron yöntemi): $x_{(k+1)} = \frac{1}{2}(x_k + a/x_k)$, $x_1 = a$. İlerleme `fprintf` ile yazdırılır (Bölüm 13.2).

Örnek 7.14 — `sqrtn(2)`

```
>> [x,iter] = sqrtn(2)
 k             x_k                rel. change
 1:  1.5000000000000000e+000   3.33e-001
 2:  1.4166666666666665e+000   5.88e-002
 3:  1.4142156862745097e+000   1.73e-003
 4:  1.4142135623746899e+000   1.50e-006
 5:  1.4142135623730949e+000   1.13e-012
 6:  1.4142135623730949e+000   0.00e+000
x =
    1.4142
iter =
     6
```

Her turda doğru basamak sayısı yaklaşık **iki katına** çıkar ($10^{-3} \to 10^{-6} \to 10^{-12} \to 0$).

Sayfa 80 — Listing 7.5, nargin, nargout

Örnek 7.15 — Listing 7.5: `sqrtn.m`

```
function [x,iter] = sqrtn(a,tol)
%SQRTN    Square root of a scalar by Newton's method.
%         X = SQRTN(A,TOL) computes the square root of the scalar
%         A by Newton's method (also known as Heron's method).
%         A is assumed to be >= 0.
%         TOL is a convergence tolerance (default EPS).
%         [X,ITER] = SQRTN(A,TOL) returns also the number of
%         iterations ITER for convergence.

if nargin < 2, tol = eps; end

x = a;
iter = 0;
xdiff = inf;
fprintf(' k             x_k                rel. change\n')

while xdiff > tol
    iter = iter + 1;
    xold = x;
    x = (x + a/x)/2;
    xdiff = abs(x-xold)/abs(x);
    fprintf('%2.0f:  %20.16e  %9.2e\n', iter, x, xdiff)
    if iter > 50
       error('Not converged after 50 iterations.')
    end
end
```

`xdiff` = göreli değişim; `xdiff = inf` döngüye ilk girişi garantiler. `fprintf` biçimleri (🔁 Java `printf`): `%2.0f` ondalıksız sayı, `%20.16e` 16 ondalıklı üstel, `%9.2e` 2 ondalıklı üstel, `\n` yeni satır. 50 turda yakınsamazsa `error` durdurur.

Örnek 7.16 — Tolerans vererek

```
>> x = sqrtn(2,1e-4);
 k             x_k                rel. change
 1:  1.5000000000000000e+000   3.33e-001
 2:  1.4166666666666665e+000   5.88e-002
 3:  1.4142156862745097e+000   1.73e-003
 4:  1.4142135623746899e+000   1.50e-006
```

4. turda değişim 1.5e-6 < 10^{-4} → durdu.

- **`nargin`** → çağrıda verilen **girdi sayısı**; verilmeyen argümanlara **varsayılan değer** atamaya yarar (burada `tol` verilmezse `eps`). 🔁 Java'da overloading ile yapılır.
- **`nargout`** → istenen **çıktı sayısı**. Burada gerek yok (`iter` her durumda hesaplanıyor), ama bazı fonksiyonlar `nargout`'a bakıp yalnızca istenen çıktıları hesaplayarak verimli çalışır (örn. Bölüm 9'daki `eig`). Listing 7.6, s. 24'teki `marks`'ın fonksiyon halidir:

Örnek 7.17

```
>> exmark = [12 0 5 28 87 3 56];
```

Sayfa 81 — marks2 ve 7.2 İsim verme ve düzenleme

Örnek 7.18 — Listing 7.6: `marks2.m`

```
function [x_sort,x_mean,x_med,x_std] = marks2(x)
%MARKS2   Statistical analysis of marks vector.
%         Given a vector of marks X,
%         [X_SORT,X_MEAN,X_MED,X_STD] = MARKS2(X) computes a
%         sorted marks list and the mean, median, and standard deviation
%         of the marks.

x_sort = sort(x);
if nargout > 1, x_mean = mean(x); end
if nargout > 2, x_med = median(x); end
if nargout > 3, x_std = std(x); end
```

Sadece istenen çıktılar hesaplanır.

Örnek 7.19 — Tek çıktı (yalnızca sıralama yapılır)

```
>> x_sort = marks2(exmark)
x_sort =
     0     3     5    12    28    56    87
```

Örnek 7.20 — Üç çıktı (std hesaplanmaz)

```
>> [x_sort,x_mean,x_med] = marks2(exmark)
x_sort =
     0     3     5    12    28    56    87
x_mean =
   27.2857
x_med =
    12
```

7.2 M-dosyalarına isim vermek ve düzenlemek

- M-dosyası adları **değişken adlarıyla aynı kurallara** uyar (s. 30): harfle başlar, harf/rakam/alt çizgi, en fazla 63 karakter. **Büyük-küçük harfe duyarlıdır** (Unix'te hep böyleydi; Windows'ta MATLAB 7 ile geldi, Ek B.5).
- Düzenleme için iki seçenek: (1) herhangi bir ASCII metin editörü (kelime işlemci kullanılıyorsa dosya **düz ASCII** olarak kaydedilmeli); (2) MATLAB'ın **Editor/Debugger**'ı (Şekil 7.2) — `edit` ya da File → New / Open ile açılır.
- Editörün özellikleri: döngü ve `if` için **otomatik girinti**, kod bloklarını **yoruma çevirme**, **sözdizimi renklendirme**, **parantez ve tırnak eşleştirme**. File → Preferences'tan kapatılıp özelleştirilebilir.
- **Blok yorum** (MATLAB 7): bir kod bloğu iki özel satır arasına alınarak yoruma çevrilir.

Sayfa 82 — Blok yorum ve 7.3 MATLAB yolu

Örnek 7.21 — Blok yorum

```
%{
<kod bloğu>
%}
```

`%{` ile `%}` arasındaki **bütün** satırlar yorumdur, başlarında `%` olmasa bile. Blok yorumlar **iç içe** olabilir (eski blok yorumu bozmadan genişletilebilir). 🔁 Java'daki `/* ... */`.

Şekil 7.2: Editor/Debugger'da `sqrtn.m`.

7.3 M-dosyalarıyla çalışmak ve MATLAB yolu

- MATLAB fonksiyonlarının bir kısmı diskteki M-dosyalarıdır, bir kısmı yorumlayıcıya **gömülüdür** (built-in).
- **Arama yolu (search path):** MATLAB'ın M-dosyalarını aradığı klasör listesi. ⚠️ Bir M-dosyası **ancak yoldaysa** kullanılabilir.
- **`path`** → yolu gösterir/ayarlar; **`addpath`** → klasör ekler; **`pathtool`** (File → Set Path) → Path browser.
- **`what`** → bulunduğun klasördeki MATLAB dosyalarını türüne göre listeler; **`what dirname`** → yoldaki `dirname` klasörünü listeler.
- **`lookfor keyword`** → M-dosyalarının **H1 satırında** arar (s. 26); **`lookfor keyword -all`** → `help`'in gösterdiği bütün yorum satırlarında arar.
- Bazı fonksiyonlarda (örn. `fminsearch`) ilk yorum bloğundan sonra kaynakça gibi ek yorumlar vardır; `help` göstermez, **`type`** gösterir.

Sayfa 83 — which, type, exist, anahtar kelimeler ve 7.4 Startup

- **`which foo`** → `foo`'nun dosya yolunu gösterir ya da `not found` der. **`which foo -all`** → aynı adlı bütün dosyaları gösterir.
- Yolda **olmayan bir script** (fonksiyon değil), `run` + M-dosyasının **tam yolu** ile çalıştırılabilir.
- **`type foo.m`** veya **`type foo`** → dosyanın içeriğini yazar (uzantısız `foo` adlı ASCII dosya varsa `type foo` onu gösterir). Önce **`more on`** yazılırsa çıktı sayfa sayfa gelir; **`more off`** kapatır.
- **İsmin kullanılıp kullanılmadığını kontrol et:** `which`; `type` (örn. `type lu` → `lu is a built-in function`); `help`; **`exist('myname')`** — çalışma alanında değişken, yolda dosya (`.m` dahil çeşitli uzantılar) ya da klasör olup olmadığını test eder: **0** eşleşme yok, **1–8** eşleşme var (`help exist`).
- **Anahtar kelimeleri** dosya/değişken adı yapma (`iskeyword`): `break, case, catch, continue, else, elseif, end, for, function, global, if, otherwise, persistent, return, switch, try, while`.
- Yoldaki bir fonksiyon ilk çağrıldığında **belleğe derlenir** (kaynakça [75], "M-File Programming"); MATLAB dosyanın değiştiğini genellikle fark edip yeniden derler. **`clear fun`** → `fun`'u bellekten siler; **`clear functions`** → bütün fonksiyonları siler.

7.4 Başlangıç (Startup)

MATLAB açılırken MATLAB kök klasörünün altındaki `toolbox\local`'deki **`matlabrc.m`**'yi çalıştırır; bu dosya varsayılanları ayarlar ve yoldaysa **`startup.m`**'yi çağırır. `startup.m` kendi varsayılan ayarlarını yapacağın ve yola klasör ekleyeceğin yerdir; en iyi **MATLAB başlangıç klasörüne** konur (değiştirmek için Help browser'da "startup directory for MATLAB" araması).

Sayfa 84 — startup.m örneği ve 7.5 Komut/fonksiyon ikiliği

Örnek 7.22 — Yazarların (kısaltılmış) `startup.m` dosyası

```
%STARTUP    Startup file.

cd d:\matlab

% Save original path, in case want to restore standard setup.
path_org = path;

mypaths = {%
'd:/matlab/matrixcomp'
'd:/matlab/gen'
'd:/matlab/misc'
'd:/matlab/book'
'd:/matlab/tools'
'd:/matlab/mats'
'd:/matlab/eigtool'};

for i=1:length(mypaths)
    addpath(mypaths{i},'-end')
end
clear i mypaths
format compact
```

`cd` çalışma klasörünü değiştirir; `path_org` orijinal yolu saklar; `{ }` **hücre dizisi** (cell array, Bölüm 18.3); `mypaths{i}` i'inci eleman; `'-end'` klasörü yolun **sonuna** ekler (MATLAB'ın kendi fonksiyonları öncelikli kalır); geçici değişkenler silinir; `format compact` ayarlanır.

7.5 Komut/fonksiyon ikiliği

Kullanıcı fonksiyonları genelde **parantezli** argümanlarla çağrılır; `type`, `what` gibi bazı gömülü fonksiyonlar ise argümanları **boşlukla** ayrılarak çağrılır. Bu bir tutarsızlık değil, **komut/fonksiyon ikiliğidir**.

Örnek 7.23 — Üç string argüman alan örnek fonksiyon

```
function comfun(x,y,z)
%COMFUN    Illustrative function with three string arguments.
disp(x), disp(y), disp(z)
```

Örnek 7.24 — Fonksiyon biçimi

```
>> comfun('ab','cd','ef')
ab
cd
ef
```

Örnek 7.25 — Komut biçimi (eşdeğer)

```
>> comfun ab cd ef
ab
cd
ef
```

Diğer çiftler (her çiftte ilki daha yaygın):

İlk biçim	İkinci biçim
`format long`	`format('long')`
`disp('Hello')`	`disp Hello`
`diary mydiary`	`diary('mydiary')`
`warning off`	`warning('off')`

⚠️ Komut biçimi **sadece string argüman alan** fonksiyonlarda kullanılmalı:

Örnek 7.26

```
>> mean 2
ans =
    50
```

Sayfa 85 — Komut biçiminin sınırları ve alıntılar

MATLAB `2`'yi **string** (`'2'`) olarak yorumlar ve `mean` bunun **ASCII koduna (50)** uygulanır. Ayrıca komut biçimi yalnızca **çıktı istenmediğinde** kullanılabilir: `x = mean 2` **hata** verir.

Alıntılar: `>> why` → "*Cleve insisted on it.*" / "*Jack knew it was a good idea.*" (MATLAB'daki şaka komutu). Kernighan & Plauger: "*Tekrar eden ifadeleri ortak bir fonksiyona yapılan çağrılarla değiştir.*" — 386-MATLAB Kullanıcı Kılavuzu (1989): MATLAB'ın gücü geniş fonksiyon setinden gelir; bir fonksiyonun gömülü mü M-dosyasında mı olduğu kullanıcı için fark etmez. Sayfa 86 boştur.

Bölüm 7 — Akılda kalması gerekenler

1. Script = komut listesi (girdi/çıktı yok); fonksiyon = `function çıktı = ad(girdi)` .
2. **Dosya adı = fonksiyon adı.** `return y;` yazılmaz.
3. Çoklu çıktı `[a,b] = f(x)` ; çıktılar sondan bırakılabilir.
4. H1 satırı ve yorumlar `help` metni olur; `%{ %}` blok yorum.
5. Fonksiyonlarda `.*` , `./` , `.^` → vektörle de çalışır.
6. `nargin` → varsayılan değer; `nargout` → sadece istenen çıktıları hesapla.
7. Fonksiyon bulunamıyorsa dosya yolda değildir (`cd` , `addpath`).
8. `mean 2` = 50: sayılarla her zaman parantez kullan.

Bölüm 8.1 — İki Boyutlu Grafikler

Alt başlıklar: **8.1.1 Temel grafikler** (s. 87–91), **8.1.2 Eksenler ve açıklamalar** (s. 91–97), **8.1.3 Bir şekilde birden fazla grafik** (s. 97–101).

Sayfa 87 — Bölüm 8 girişi ve 8.1.1 Temel grafikler

- MATLAB'ın grafik özellikleri güçlü ve çok yönlüdür; görünüm büyük ölçüde özelleştirilebilir. Bu bölüm 2B ve 3B veriler için en çok kullanılan araçların **temel kullanımını** anlatır; iç yapı Bölüm 17'de (Handle Graphics). Son bölüm anlatılmayanlara değinir.
- Odak: grafikleri **komut satırından veya M-dosyalarından** üretmek. Var olan şekiller **Plot Editor** ile fareyle de düzenlenebilir (`help plotedit`, Tools menüsü ve araç çubuğu).
- Kitaptaki şekiller pencerenin olduğu gibi kaydedilmiş halidir (Bölüm 8.4). Kitap siyah-beyaz; ekranda çoğu **renklidir**.

`plot` → basit **"noktaları birleştir"** x-y grafiği.

Örnek 8.1

```
>> x = [1.5  2.2  3.1  4.6  5.7  6.3  9.4];
>> y = [2.3  3.9  4.3  7.2  4.5  6.1  1.1];
>> plot(x,y)
```

(x(i), y(i)) noktaları sırayla birleştirilir (Şekil 8.1 sol). MATLAB bir **şekil penceresi** açar (açık değilse); eksen aralıkları, işaret aralıkları, çizgi rengi ve türü için **varsayılan** değerler kullanılır.

`plot(x,y,string)` → `string` en fazla üç öğe birleştirir: **renk, işaretçi (marker), çizgi stili**.

Örnek 8.2

```
plot(x,y,'r*--')
```

Her noktada kırmızı yıldız, noktalar kırmızı kesikli çizgiyle birleşik.

Sayfa 88 — Tablo 8.1, stil örnekleri, matris argümanlar

Örnek 8.3

```
plot(x,y,'y+')
```

Sarı artı işaretleri; **çizgi yok** (çizgi stili verilmedi).

Şekil 8.1: Solda varsayılan, sağda varsayılan olmayan stil.

Tablo 8.1 — `plot` seçenekleri:

Renk		İşaretçi		Çizgi stili	
`r`	Kırmızı	`o`	Daire	`-`	Düz (varsayılan)
`g`	Yeşil	`*`	Yıldız	`--`	Kesikli
`b`	Mavi	`.`	Nokta	`:`	Noktalı
`c`	Camgöbeği	`+`	Artı	`-.`	Çizgi-nokta
`m`	Eflatun	`x`	Çarpı		
`y`	Sarı	`s`	Kare		
`k`	**Siyah**	`d`	Elmas		
`w`	Beyaz	`^`	Yukarı üçgen		
		`v`	Aşağı üçgen		
		`>`	Sağa üçgen		
		`<`	Sola üçgen		
		`p`	Beş köşeli yıldız		
		`h`	Altı köşeli yıldız		

⚠️ Siyah = `k` (çünkü `b` mavi).

Örnek 8.4 — Şekil 8.1 sağ

```
plot(x,y,'kd:')
```

Siyah, elmas işaretçi, noktalı çizgi.

Örnek 8.5 — Öğelerin sırası önemsiz (ikisi eşdeğer)

```
plot(x,y,'ms--')
plot(x,y,'s--m')
```

Örnek 8.6 — Birden fazla veri seti

```
plot(x,y,'g-',b,c,'r--')
```

x–y yeşil düz, b–c kırmızı kesikli; üst üste çizilir.

- **Matris argümanlar:** x m-vektör, Y m×n ise `plot(x,Y)` → Y'nin **her sütunu** x'e karşı. X ve Y ikisi de m×n ise `plot(X,Y)` → karşılık gelen sütunlar. Karmaşık sayılar verilirse sanal kısımlar genelde **yok sayılır** — tek istisna:

Sayfa 89 — Tek argüman, ek özellikler, loglog

- **Tek argüman:** Y karmaşıksa `plot(Y)` = `plot(real(Y),imag(Y))`. Y gerçekse `plot(Y)` sütunları **indekslerine** (1, 2, 3, ...) karşı çizer.
- **Ek özellikler** ('Ad', değer çiftleri): **`LineWidth`** (varsayılan 0.5 punto) ve **`MarkerSize`** (varsayılan 6 punto); 1 punto = 1/72 inç.

Örnek 8.7

```
plot(x,y,'LineWidth',2)
```

Örnek 8.8

```
plot(x,y,'p','MarkerSize',10)
```

İçi olan işaretçiler için `MarkerEdgeColor` (kenar) ve `MarkerFaceColor` (iç) Tablo 8.1 renkleriyle ayarlanır.

Örnek 8.9

```
plot(x,y,'o','MarkerEdgeColor','m')
```

Dairelerin kenarı eflatun.

Örnek 8.10 — Şekil 8.2 sol

```
plot(x,y,'m--^','LineWidth',3,'MarkerSize',5)
```

Örnek 8.11 — Şekil 8.2 sağ

```
plot(x,y,'--rs','MarkerSize',20,'MarkerFaceColor','g')
```

Büyük (20 punto), içi yeşil kırmızı kareler, kesikli çizgi. Varsayılan değerler Tablo 8.2'de.

`loglog` → iki eksen de **logaritmik**. Kuvvet ilişkileri ($y = c \cdot x^k$) **düz çizgi** olarak görünür (eğim = k). $|1 + h + h^2/2 - \exp(h)|$ hatası küçük h için **h^3'ün katı** gibi davranır → log-log grafikte **eğimi 3** olan doğruya yakın olmalı; doğrulamak için kesikli bir referans çizgisi de çizilir.

Örnek 8.12 — Şekil 8.3 (sayfa 89–90)

```
h = 10.^[0:-1:-4];
taylerr = abs((1+h+h.^2/2) - exp(h));
loglog(h,taylerr,'-',h,h.^3,'--')
xlabel('h')
ylabel({'Absolute value','of error'})
title('Error in quadratic Taylor series approximation to exp(h)')
box off
```

h = [1 0.1 0.01 0.001 0.0001]; hata düz, h^3 kesikli; iki çizgi **paralel** (eğim 3).

Sayfa 90 — Tablo 8.2, etiketler, hold

Tablo 8.2 — Bazı özelliklerin varsayılanları:

Özellik	Varsayılan
`LineWidth`	0.5
`MarkerSize`	6
`MarkerEdgeColor`	auto
`MarkerFaceColor`	none
`FontSize`	10
`FontAngle`	normal

- `title`, `xlabel`, `ylabel` → metni grafiğin **üstüne**, **x eksenine**, **y eksenine** yazar. Çok satırlı y etiketi bir **hücre dizisiyle** oluşturuldu (her string bir satır; Bölüm 18.3).
- `box off` → grafiğin etrafındaki kutuyu kaldırır, sadece x ve y eksenleri kalır.
- `loglog`'a pozitif olmayan veri verilirse uyarı verir ve sadece pozitif verileri çizer. `semilogx` → yalnız x ekseni, `semilogy` → yalnız y ekseni logaritmik.

- **`hold on`** → sonraki grafikler mevcut grafiğin **üstüne** çizilir; **`hold off`** → her yeni grafik **sıfırdan** başlar. **Varsayılan: `hold off`.**

Sayfa 91 — Pencereler, Tablo 8.3, 8.1.2 Eksenler

- **`clf`** → mevcut şekli temizler; **`close`** → kapatır; **`figure`** → yeni pencere; **`figure(n)`** → n numaralı pencereyi etkin yapar; **`close all`** → bütün pencereleri kapatır.
- Şekil çizildikten sonra araç çubuğu ve Tools menüsüyle fareyle değiştirilebilir; örneğin bir bölgeye **zoom** yapılabilir (`help zoom`).

Tablo 8.3 — Eksen komutları:

Komut	İşlevi
`axis([xmin xmax ymin ymax])`	x ve y sınırlarını ayarla
`axis auto`	Varsayılan sınırlara dön
`axis equal`	x, y, z eksenlerinde birim uzunlukları eşitle
`axis off`	Eksenleri kaldır
`axis square`	Eksen kutusunu kare yap
`axis tight`	Sınırları verinin aralığına ayarla
`xlim([xmin xmax])`	x sınırları
`ylim([ymin ymax])`	y sınırları

8.1.2 Eksenler ve açıklamalar

`axis off` eksenleri kaldırır; `axis equal` en-boy oranını 1 yapar (daire **elips değil daire** görünür); `axis square` kutuyu kare yapar.

Örnek 8.13 — Şekil 8.4 sol

```
plot(fft(eye(17))), axis equal, axis square
```

`fft(eye(17))` birim çember üzerinde karmaşık sayılardan oluşan bir matris üretir; tek karmaşık argümanlı `plot` gerçek kısmı x'e, sanal kısmı y'ye çizer → yıldız deseni.

Örnek 8.14 — Şekil 8.4 sağ (grafik birim çemberde kaldığı için eksen gereksiz)

```
plot(fft(eye(17))), axis equal, axis off
```

Bu ilginç resmin anlamı kaynakça [79]'da.

- `axis([xmin xmax ymin ymax])` sınırları ayarlar; `axis auto` varsayılana döner. Bir sınırı MATLAB'ın seçmesi için **`-inf`** veya **`inf`**: örn. `axis([-1 1 -inf 0])`. Tek eksen için `xlim`, `ylim`.

Örnek 8.15 — [0,3] üzerinde $1/(x-1)^2 + 3/(x-2)^2$

```
x = linspace(0,3,500);
plot(x,1./(x-1).^2 + 3./(x-2).^2)
grid on
```

`grid on` → eksen işaretlerinden uzanan açık renkli yatay/dikey ızgara.

Sayfa 92 — ylim ve episikloid

Şekil 8.5: x = 1 ve 2'deki **tekillikler** yüzünden sol grafik bilgi vermez (sadece iki sivri çizgi).

Örnek 8.16 — İlginç kısma odaklanmak (Şekil 8.5 sağ)

```
ylim([0 50])
```

Örnek 8.17 — Episikloid: x(t) = (a+b)cos t − b·cos((a/b+1)t), y(t) = (a+b)sin t − b·sin((a/b+1)t), 0 ≤ t ≤ 10π, a = 12, b = 5 (sayfa 92–93)

```
a = 12; b = 5;
t = 0:0.05:10*pi;
x = (a+b)*cos(t) - b*cos((a/b+1)*t);
y = (a+b)*sin(t) - b*sin((a/b+1)*t);

plot(x,y)
axis equal
axis([-25 25 -25 25])
grid on

title('Epicycloid: a=12, b=5')
xlabel('x(t)'), ylabel('y(t)')
```

Parametrik eğri (Şekil 8.6). `axis` sınırları eğrinin çevresinde boşluk kalacak şekilde seçilmiş.

Sayfa 93 — Legendre polinomları ve legend

Örnek 8.18 — 1–4. derece Legendre polinomları, `legend` ile (Şekil 8.7; sayfa 93–94)

```
x = -1:.01:1;
p1 = x;
p2 = (3/2)*x.^2 - 1/2;
p3 = (5/2)*x.^3 - (3/2)*x;
p4 = (35/8)*x.^4 - (15/4)*x.^2 + 3/8;

plot(x,p1,'r:',x,p2,'g--',x,p3,'b-.',x,p4,'m-')
box off

legend('\it n=1','\it n=2','\it n=3','\it n=4','Location','SouthEast')
xlabel('x','FontSize',12,'FontAngle','italic')
ylabel('P_n','FontSize',12,'FontAngle','italic','Rotation',0)
title('Legendre Polynomials','FontSize',14)
text(-.6,.7,'(n+1)P_{n+1}(x) = (2n+1)x P_n(x) - n P_{n-1}(x)',...
     'FontSize',12,'FontAngle','italic')
```

`legend` sağ alta açıklama kutusu koyar; `\it` italik; `P_n` → P_n (alt indis); `'Rotation',0` y etiketini yatay yazar; `text` grafiğe (−0.6, 0.7) noktasına formül yazar. (Polinomların özellikleri: kaynakça [14].)

Sayfa 94 — legend konumları, text, TeX

- **`legend('string1',...,'stringn')`** → her string'in yanına ilgili grafiğin renk/işaretçi/çizgi stilini koyar. Varsayılan konum **sağ üst** (kuzeydoğu). Konum: `legend(...,'Location',location)`:

Değer	Anlamı
`'North'`	Grafik kutusunun içinde, üstte
`'NorthWest'`	İçeride, sol üstte
`'NorthOutside'`	Grafik kutusunun dışında, üstte
`'Best'`	Veriyle en az çakışacak yer (otomatik)
`'BestOutside'`	Grafiğin dışında, en az boş alan bırakacak yer

Kısaltmalar: `'N'`, `'NW'` gibi. Kutu fareyle sürüklenebilir; diğer seçenekler `doc legend`.

- **`text(x,y,'string')`** → metni (x,y) noktasına yazar. **`gtext`** → yeri fareyle seçtirir.
- `ylabel` ve `text`'teki metinler **TeX** gösterimini kullanır: Yunan harfleri, matematik sembolleri, yazı tipleri, üst/alt indisler (`_` alt, `^` üst; birden fazla karakter için `{ }`). Liste Tablo 8.4'te; tamamı `doc text_props` → string.

Sayfa 95 — Tablo 8.4, texlabel, LaTeX

Tablo 8.4 — Metinlerde desteklenen bazı TeX komutları:

Yunan (küçük)		Yunan (büyük)		Sembol		Sembol	
α	`\alpha`	Γ	`\Gamma`	≈	`\approx`	≠	`\neq`
β	`\beta`	Δ	`\Delta`	∘	`\circ`	⊗	`\otimes`
γ	`\gamma`	Θ	`\Theta`	≥	`\geq`	∂	`\partial`
⋮	⋮	⋮	⋮	ℑ	`\Im`	±	`\pm`
ω	`\omega`	Ω	`\Omega`	∈	`\in`	ℜ	`\Re`
				∞	`\infty`	~	`\sim`
				∫	`\int`	√	`\surd`
				≤	`\leq`		

Yazı tipleri: Normal `\rm`, **Kalın** `\bf`, *İtalik* `\it`.

Örnek 8.19 — Süslü parantez yazı tipi ve indislerin kapsamını belirler

```
title('{\itItalic} Normal {\bfBold} \int_{-\infty}^\infty')
```

Başlık: *Italic* Normal **Bold** ∫₋∞^∞. ⚠️ TeX'ten farklı olarak, yazı tipi komutundan sonra bırakılan **boşluk yazdırılır**.

Örnek 8.20 — `texlabel`: TeX bilmeden MATLAB ifadesi tarzında yazmak (iki satır eşdeğer)

```
text(5,5,'\alpha^{3/2}+\beta^{12}-\sigma_i')
text(5,5,texlabel('alpha^(3/2)+beta^12-sigma_i'))
```

- Legendre örneğinde `xlabel`, `ylabel`, `title`, `text` metinlerinin boyut ve açısı **`FontSize`** ve **`FontAngle`** ile ayarlandı (varsayılan 10 ve normal). **`legend` bu argümanları kabul etmediği için** italik için TeX (`\it`) kullanıldı. Belgeye/sunuma konacak grafiklerde yazıyı büyütmek okunabilirliği artırır. y etiketi **`Rotation`** ile döndürüldü.

- **LaTeX yorumlayıcısı** (MATLAB 7): `interpreter` özelliği **`latex`** yapılınca LaTeX'in matematik dizgi özellikleri kullanılır; etiketler, başlık ve `text` üzerinde çok daha fazla kontrol sağlar ve `psfrag` gibi tekniklere gerek bırakmaz.

Sayfa 96 — LaTeX örneği ve fill

Örnek 8.21 — x = [1 2 ... 10]ᵀ'nin p-normu, LaTeX metinli (Şekil 8.8)

```
n = 10; m = 100;
x = 1:n;
y = zeros(m,1);

pvals = linspace(1,10,m);
for i = 1:m
    y(i) = norm(x,pvals(i));
end

plot(pvals,y,'LineWidth',2)
ylim([0 inf])
options = {'Interpreter','latex','FontSize',18};
ylabel('$\|x\|_p$',options{:},'HorizontalAlignment','right')
xlabel('$p$',options{:})
title(['\slshape Vector $p$-norm, for $x =' ...
       '[1~2~\dots~' int2str(n) ']^T$'], options{:})

s = '$$\|x\|_p = \biggl(\sum_{i=1}^n|x_i|^p\biggr)^{1/p}$$';
text(options{:},'String',s,'Position',[3 40])
```

p = 1'de norm 55 (1+...+10), p büyüdükçe en büyük eleman 10'a yaklaşır. **`options` hücre dizisi** `'Interpreter','latex','FontSize',18`'i tekrar tekrar yazmamak için; `options{:}` listeyi açar. `$...$` LaTeX matematik modu, `int2str(n)` sayıyı metne çevirir.

`fill(x,y,[r g b])` → köşeleri (x(i), y(i)) olan **çokgeni boyar**; noktalar sırayla alınır, son köşe ilkine bağlanır. Renk `[r g b]` ile: kırmızı, yeşil, mavi miktarları **[0,1]** aralığında. `[0 1 0]` saf yeşil, `[1 0 1]` eflatun.

Sayfa 97 — Bézier eğrisi, annotation, 8.1.3 subplot

Eşit miktarda r, g, b → **gri**; siyah `[0 0 0]` ile beyaz `[1 1 1]` arasında değişir.

Kübik Bézier eğrisi: $p(u) = (1-u)^3P_1 + 3u(1-u)^2P_2 + 3u^2(1-u)P_3 + u^3P_4$, $0 \le u \le 1$. P_1...P_4 kontrol noktaları; P matrisi P_j'yi j'inci sütununda saklar. `fill(P(1,:),P(2,:),[.8 .8 .8])` kontrol çokgenini açık griye boyar; `Curve` sütunları eğri üzerindeki sık noktalardır.

Örnek 8.22 — Şekil 8.9

```
P = [0.1 0.3 0.7 0.8;
     0.3 0.8 0.6 0.1];
plot(P(1,:),P(2,:),'*')
axis([0 1 0 1])
hold on

u = 0:.01:1;
umat = [(1-u).^3; 3.*u.*(1-u).^2; 3.*u.^2.*(1-u); u.^3];
Curve = P*umat;
fill(P(1,:),P(2,:),[.8 .8 .8])
plot(Curve(1,:),Curve(2,:),'--')

text(0.35,0.35,'control polygon')
text(0.05,0.3,'P_1')
text(0.25,0.8,'P_2')
text(0.72,0.6,'P_3')
text(0.82,0.1,'P_4')
hold off
```

`umat` 4×101, `P` 2×4 → `Curve = P*umat` 2×101: eğri tek **matris çarpımıyla** hesaplanır. `hold on` ile hepsi aynı grafikte.

- **`annotation`** → çizgi, çeşitli oklar, dikdörtgen ve elips gibi açıklama nesneleri. Güçlü ama kullanımı zor: yer **normalleştirilmiş koordinatlarla** verilir (şeklin sol altı (0,0), sağ üstü (1,1)). En kolayı pencerenin etkileşimli araçlarıyla oluşturmak (kaynakça [76]).

8.1.3 Bir şekilde birden fazla grafik

`subplot(mnp)` veya **`subplot(m,n,p)`** → pencereyi **m×n** bölgeye ayırır (her birinin kendi ekseni var); sonraki komutlar **p'inci** bölgeye uygulanır. Sayma: **önce birinci satır boyunca**, sonra ikinci satır... Örn. `subplot(425)` → 4×2 ızgaranın 5. bölgesi (3. satırın ilki); ardından `subplot(427)` → (4,1) bölgesi.

```
┌───┬───┐
│ 1 │ 2 │
├───┼───┤
│ 3 │ 4 │
├───┼───┤
│ 5 │ 6 │   ← subplot(425)
├───┼───┤
│ 7 │ 8 │   ← subplot(427)
└───┴───┘
```

Sayfa 98 — fplot

Şekil 8.9: Gri kontrol çokgeni, köşelerde P_1–P_4, kesikli Bézier eğrisi.

`fplot` → fonksiyonu yeterli sayıda noktada **uyarlanabilir** biçimde hesaplayıp çizer (x vektörünü sen oluşturmazsın).

Örnek 8.23 — Şekil 8.10

```
subplot(221), fplot('exp(sqrt(x)*sin(12*x))',[0 2*pi])
subplot(222), fplot('sin(round(x))',[0 10],'--')
subplot(223), fplot('cos(30*x)/x',[0.01 1 -15 20],'-.')
subplot(224), fplot('[sin(x),cos(2*x),1/(1+x)]',[0 5*pi -1.5 1.5])
```

1) exp(√x·sin 12x), 0 ≤ x ≤ 2π. 2) Varsayılan düz çizgi yerine **kesikli** `'--'`. 3) `[0.01 1 -15 20]` → **hem x hem y sınırları** (0.01 ≤ x ≤ 1, −15 ≤ y ≤ 20), `'-.'` çizgi-nokta. 4) **Tek çağrıda birden fazla fonksiyon.**

Genel kalıp: `fplot(fun,lims,tol,N,'LineSpec',p1,p2,...)`

Argüman	Anlamı
`fun`	Çizilecek fonksiyon: string veya function handle (Bölüm 10.1)
`lims`	x ve/veya y sınırları
`tol`	Göreli hata toleransı; varsayılan 2×10^{-3} (%0.2 doğruluk)
`N`	En az N+1 nokta kullanılır
`LineSpec`	Çizgi türü
`p1, p2, ...`	`fun`'a geçirilen parametreler; `fun`'ın girdileri `x,p1,p2,...` olmalı

Sayfa 99 — fplot ile cheby, düzensiz ızgaralar

Şekil 8.10: `subplot` ve `fplot` örneği.

`tol`, `N` ve `'LineSpec'` **herhangi bir sırada** verilebilir; varsayılanı kullanmak için yerine `[]` yazılır.

Örnek 8.24 — Listing 7.4'teki `cheby` ile (Şekil 8.11)

```
subplot(211), fplot(@cheby,[-1 1],[],[],[],5)
subplot(212), fplot(@cheby,[-1 1],[],[],[],35)
```

Üstte ilk **5**, altta ilk **35** Chebyshev polinomu. `@cheby` = function handle (Bölüm 10.1); `[],[],[]` = tol, N, LineSpec varsayılan; 5 / 35 = `cheby`'nin `p` parametresi.

`subplot`'u farklı ızgara düzenleriyle çağırarak **düzensiz** yerleşim yapılabilir.

Örnek 8.25 — Şekil 8.12

```
x = linspace(0,15,100);
subplot(2,2,1), plot(x,sin(x))
subplot(2,2,2), plot(x,round(x))
subplot(2,1,2), plot(x,sin(round(x)))
```

İlk ikisi 2×2 ızgaranın üst bölgeleri; üçüncüsü 2×1 ızgaranın 2. bölgesi = **alt yarının tamamı**.

Örnek 8.26 — Üçüncü argüman bölge **vektörü** olabilir (son satırın eşdeğeri)

```
subplot(2,2,3:4), plot(x,sin(round(x)))
```

En popüler 2B çizim fonksiyonları Tablo 8.5'tedir (bazıları Bölüm 8.3'te anlatılır).

Sayfa 100–101 — Şekil 8.11, 8.12 ve Tablo 8.5

Şekil 8.11: Üstte ilk 5, altta ilk 35 Chebyshev polinomu (`fplot` + `cheby`). **Şekil 8.12:** `subplot` ile düzensiz ızgara: sol üstte sinüs, sağ üstte merdiven (`round`), altta geniş basamaklı sinüs.

Tablo 8.5 — 2B çizim fonksiyonları (s. 101):

Fonksiyon	Açıklama	Fonksiyon	Açıklama
`plot`	Basit x-y grafiği	`area`	İçi dolu alan grafiği
`loglog`	İki ekseni logaritmik	`bar`	Çubuk grafik
`semilogx`	x ekseni logaritmik	`barh`	Yatay çubuk grafik
`semilogy`	y ekseni logaritmik	`hist`	Histogram
`plotyy`	Solda ve sağda iki y ekseni	`pie`	Pasta grafik
`polar`	Kutupsal koordinatlar	`comet`	Kuyruklu yıldız gibi hareketli x-y grafiği
`fplot`	Fonksiyon çizici	`errorbar`	Hata çubuklu grafik
`ezplot`	Kullanımı kolay fonksiyon çizici	`quiver`	Ok (hız vektörü) grafiği
`ezpolar`	`polar`'ın kolay hali	`scatter`	Saçılım grafiği
`fill`	İçi dolu çokgen	`stairs`	Merdiven grafiği

Bölüm 8.1 — Akılda kalması gerekenler

1. `plot(x,y,'r*--')` : renk + işaretçi + çizgi stili, sıra önemsiz; siyah = `k` .
2. Ek özellikler 'Ad', değer çiftleriyle: `LineWidth` , `MarkerSize` , `MarkerEdgeColor` , `MarkerFaceColor` , `FontSize` , `FontAngle` , `Rotation` .
3. `hold on/off` (varsayılan off), `figure` , `clf` , `close` .
4. `axis([...])` , `axis equal/square/off/tight` , `xlim` , `ylim` , `grid on` , `box off` .
5. `title` , `xlabel` , `ylabel` , `legend(...,'Location',...)` , `text` ; TeX: `_` alt, `^` üst, `\alpha` , `\it` .
6. `loglog` / `semilogx` / `semilogy` ; `fill(x,y,[r g b])` .
7. `subplot(m,n,p)` : satır satır sayılır; `fplot(fun,lims,...)` .

Bölüm 8.2 — Üç Boyutlu Grafikler

Kitap sayfa 101–110. Konular: `plot3`, kontur grafikleri (`ezcontour`, `contour`, `clabel`, `meshgrid`), yüzey grafikleri (`mesh`, `meshc`, `surf`, `surfc`, `waterfall`, `meshz`), renk ve gölgeleme (`colorbar`, `colormap`, `shading`), bakış açısı (`view`), NaN ile eksik veri.

Sayfa 101 — plot3

`plot3`, `plot`'un 3B karşılığıdır: `plot3(x,y,z)` → (x(i), y(i), z(i)) noktalarını sırayla birleştiren "noktaları birleştir" eğrisi.

Örnek 8.27 — Şekil 8.13

```
t = -5:.005:5;
x = (1+t.^2).*sin(20*t);
y = (1+t.^2).*cos(20*t);
z = t;

plot3(x,y,z)
grid on
FS = 'FontSize';
xlabel('x(t)',FS,14), ylabel('y(t)',FS,14)
zlabel('z(t)',FS,14,'Rotation',0)
title('\it{plot3 example}',FS,14)
```

- t boyunca dönen (sin 20t, cos 20t) ve yarıçapı 1 + t² ile büyüyen bir **sarmal**: ortası dar, uçları geniş, kum saatine benzeyen bir şekil.
- `xlabel`, `ylabel`, `title` önceki bölümden; **`zlabel`** z eksenine etiket. `FS = 'FontSize'` uzun özellik adını kısaltmak için bir değişken. Başlıkta TeX `\it` ile italik yazı.
- **Renk, işaretçi ve çizgi stili `plot` ile aynı şekilde** verilir. Örneğin `plot3(x,y,z,'rx--')` → kırmızı kesikli çizgi ve her noktada çarpı.
- ⚠️ 3B grafiklerde varsayılan **`box off`**; `box on` grafiği çevreleyen bir kutu ekler.

Sayfa 102 — Kontur grafikleri: ezcontour, contour, meshgrid

Şekil 8.13: `plot3` ile çizilen 3B sarmal.

Kontur (eş yükselti) grafiği, bir yüzeyin **aynı yükseklikteki noktalarını birleştiren çizgileri** gösterir (haritadaki eş yükselti eğrileri gibi). Basit kontur çizimi için **`ezcontour`** vardır.

Örnek 8.28 — Şekil 8.14 (üstte `ezcontour`, altta `contour`)

```
subplot(211)
ezcontour('sin(3*y-x^2+1)+cos(2*y^2-2*x)',[-2 2 -1 1]);

x = -2:.01:2; y = -1:.01:1;
[X,Y] = meshgrid(x,y);
Z = sin(3*Y-X.^2+1)+cos(2*Y.^2-2*X);

subplot(212)
contour(x,y,Z,20)
```

- `ezcontour`: sin(3y − x² + 1) + cos(2y² − 2x) fonksiyonunu −2 ≤ x ≤ 2, −1 ≤ y ≤ 1 aralığında çizer; kontur seviyeleri **otomatik** seçilir.
- Altta daha genel **`contour`** kullanılıyor:
- `x = -2:.01:2`, `y = -1:.01:1` → uygun aralıkta sık noktalar.
- **`[X,Y] = meshgrid(x,y)`** → X'in **her satırı** x'in kopyası, Y'nin **her sütunu** y'nin kopyası olan matrisler. (`meshgrid`, MATLAB'ın 3B çizim araçları için veri hazırlarken **çok faydalıdır**.)
- Z, X ve Y üzerinde **dizi işlemleriyle** (`.^`) hesaplanır; böylece **Z(i,j)** = fonksiyonun **(x(j), y(i))** noktasındaki değeri. Bu, `contour`'un istediği biçimdir.
- `contour(x,y,Z,20)` → Z'yi x-y düzleminin üstündeki **yükseklikler** olarak alır (aralıklar x ve y ile verilir). Son argüman **20 kontur seviyesi** ister; verilmezse sayıyı MATLAB seçer.

💡 `meshgrid` mantığı küçük örnekle: `[X,Y] = meshgrid(1:3, 10:20:50)` →

```
X =  1  2  3        Y = 10 10 10
     1  2  3            30 30 30
     1  2  3            50 50 50
```

Her (X(i,j), Y(i,j)) çifti ızgaranın bir noktası olur.

Sayfa 103 — clabel ile etiketli konturlar; mesh

Örnek 8.29 — "Altı tümsekli deve sırtı" fonksiyonu, `clabel` ile (Şekil 8.15)

```
[X,Y] = meshgrid(-3:.05:3, -1.5:.025:1.5);
Z = 4*X.^2 - 2.1*X.^4 + X.^6/3 + X.*Y - 4*Y.^2 + 4*Y.^4;
cvals = [-2:.5:2 2.3 3:5 6:2:10];
[C,h] = contour(X,Y,Z,cvals);
clabel(C,h,cvals([1:2:9 10 11 14 16]))
xlabel('x'), ylabel('y')
title('Six hump camel back function','Fontsize',16)
```

- Fonksiyonun birçok **maksimumu, minimumu ve eyer noktası** var. MATLAB'ın varsayılan seviyeleri güzel bir resim vermediği için seviyeler (deneme yanılmayla) **`cvals`** vektöründe elle verildi.
- **`[C,h] = contour(...)`** → `C` kontur verisi, `h` grafik nesnesine bir **handle**.
- **`clabel(C,h,seviyeler)`** → üçüncü argümanda verilen seviyelerin konturlarına **sayı etiketleri** ekler. Seviye verilmeyebilir, ama bu örnekte bütün konturları etiketlemek kalabalık bir grafik verir.
- **`clabel(C,h,'manual')`** → etiketlenecek konturları **fareyle** seçtirir: kontura tıkla, bitince Enter.
- `h` verilmezse etiketler **döndürülmeden**, konturun yanına konur ve kontur bir **artı işaretiyle** işaretlenir.

`mesh` → `contour` gibi veri alır ve **tel kafes (wireframe) yüzey** çizer. **`meshc`** → yüzeyin **altına kontur** grafiği ekler.

Aşağıdaki örnek (Şekil 8.16) sin(y² + x) − cos(y − x²), 0 ≤ x, y ≤ π yüzeyini kullanır: - 1. alt grafik `mesh(Z)`: x, y verilmediği için eksenlerde **satır ve sütun indeksleri** kullanılır. - 2. alt grafik `meshc(Z)`'nin etkisini gösterir. - 3. alt grafikte `mesh(x,y,Z)`: eksen işaretleri **x ve y değerlerine** karşılık gelir. `axis([0 pi 0 pi -5 5])` → 0 ≤ x, y ≤ π ve −5 ≤ z ≤ 5. - 4. alt grafikte yine `mesh(Z)`, ardından **`hidden off`** → normalde gizlenen arka çizgiler de gösterilir.

Sayfa 104 — mesh örneği; surf, surfc, colorbar, shading, waterfall

Şekil 8.15: Seçilen konturları sayılarla etiketlenmiş deve sırtı fonksiyonu.

Örnek 8.30 — Şekil 8.16

```
x = 0:.1:pi; y = 0:.1:pi;
[X,Y] = meshgrid(x,y);
Z = sin(Y.^2+X)-cos(Y-X.^2);

subplot(221)
mesh(Z)

subplot(222)
meshc(Z)

subplot(223)
mesh(x,y,Z)
axis([0 pi 0 pi -5 5])

subplot(224)
mesh(Z)
hidden off
```

- `surf`, `mesh`'ten farklı olarak **içi dolu (renkli) yüzey** çizer; `surfc` altına kontur ekler.
- Sıradaki örnekte MATLAB'ın `membrane` fonksiyonu kullanılır: L şeklindeki bir zarın ilk **özfonksiyonunu** döndürür (MATLAB logosundaki şekil). Şekil 8.17'nin üst satırı `surf` ve `surfc`'nin etkisini gösterir.
- (2,1) grafiği `colorbar` ile bir **renk skalası** gösterir. Mevcut şeklin renk haritası `colormap` ile ayarlanır (`doc colormap`).
- (2,1) grafiğinde `shading flat` yüzeydeki **ızgara çizgilerini kaldırır**; diğer seçenek `shading interp` rengi her parça üzerinde **interpolasyonla** değiştirir.
- (2,2) grafiğinde `waterfall`: `mesh`'e benzer, ama **sütun yönündeki** tel çizgiler kaldırılmıştır (şelale görünümü).

Sayfa 105 — surf örneği, renkler ve view

Şekil 8.16: `mesh` ve `meshc` ile yüzey grafikleri (dört alt grafik).

Örnek 8.31 — Şekil 8.17

```
Z = membrane; FS = 'FontSize';
subplot(221), surf(Z), title('\bf{surf}',FS,14)
subplot(222), surfc(Z), title('\bf{surfc}',FS,14), colorbar
subplot(223), surf(Z), shading flat
              title('\bf{surf} shading flat',FS,14)
subplot(224), waterfall(Z), title('\bf{waterfall}',FS,14)
```

`\bf` TeX ile kalın başlık.

- Yüzeyin rengi normalde **x-y düzleminin üstündeki yüksekliği** gösterir. Ama `mesh`, `surf` ve benzerleri `mesh(x,y,Z,W)` biçiminde de kullanılabilir: renkler **W dizisine** göre belirlenir. Böylece yüzeyin başka bir özelliği gösterilebilir ya da bağımsız bir renk deseni uygulanabilir.

Bakış açısı — `view`: Şekil 8.13, 8.16 ve 8.17 MATLAB'ın **varsayılan bakış açısını** kullanır. Değiştirmek için: - **`view(a,b)`** → z ekseni etrafında saat yönünün tersine **a derece** dönüş (azimut) ve **b derece** dikey yükseklik (elevasyon). - Varsayılan: **`view(-37.5,30)`**. - **`view(2)`** = `view(0,90)` → yüzeye **tepeden bakan** 2B görünüm. - Şekil penceresindeki **Rotate 3D** aracı ile eksen alanında tıklayıp sürükleyerek bakış açısı fareyle değiştirilebilir.

2B bir grafiğe de **3B açıdan** bakılabilir: `view` ile bir açı vererek ya da sadece **`view(3)`** yazarak.

Örnek 8.32 — Şekil 8.18 (Bölüm 8.1'deki yıldız deseninin 3B görünümü)

```
plot(fft(eye(17))); view(3); grid
```

Fraktal manzara: Listing 8.1'deki **özyinelemeli** (kendini çağıran) `land` fonksiyonu, **rastgele orta nokta yer değiştirme** (random midpoint displacement) algoritmasının bir çeşidiyle fraktal bir manzara üretir (kaynakça [90, Bölüm 7.6]; Şekil 8.19). Özyineleme Bölüm 10.9'da ayrıntılı. Temel adım: sadece **dört köşesi** sıfırdan farklı olan N×N bir matriste (1,d), (d,1), (d,d), (d,N) ve (N,d) konumlarını (d = (N+1)/2) şöyle doldurmak:

Sayfa 106 — Şekil 8.17 ve 8.18

Şekil 8.17: `surf`, `surfc` (yanında renk skalası), `surf` + `shading flat` ve `waterfall` ile `membrane` yüzeyi. **Şekil 8.18:** 2B yıldız deseninin 3B kutu içinde, yatay bir disk gibi görünümü.

Sayfa 107 — land fonksiyonu

Örnek 8.33 — Listing 8.1: `land.m`

```
function B = land(A)
%LAND      Fractal landscape.
%          B = LAND(A) generates a random fractal landscape
%          represented by B, where A is a square matrix of
%          dimension N = 2^n + 1 whose four corner elements
%          are used as input parameters.

N = size(A,1);
d = (N+1)/2;
level = log2(N-1);
scalef = 0.05*(2^(0.9*level));

B = A;

B(d,d) = mean([A(1,1),A(1,N),A(N,1),A(N,N)]) + scalef*randn;
B(1,d) = mean([A(1,1),A(1,N)]) + scalef*randn;
B(d,1) = mean([A(1,1),A(N,1)]) + scalef*randn;
B(d,N) = mean([A(1,N),A(N,N)]) + scalef*randn;
B(N,d) = mean([A(N,1),A(N,N)]) + scalef*randn;

if N > 3
   B(1:d,1:d) = land(B(1:d,1:d));
   B(1:d,d:N) = land(B(1:d,d:N));
   B(d:N,1:d) = land(B(d:N,1:d));
   B(d:N,d:N) = land(B(d:N,d:N));
end
```

Köşeleri w, x, y, z olan matriste:

```
w     .     x              w      (w+x)/2      x
.     .     .      →    (w+y)/2  (w+x+y+z)/4  (x+z)/2    + gürültü
y     .     z              y      (y+z)/2      z
```

- Yeni her elemana `randn`'ın bir katı (**gürültü**) eklenir.
- İşlem, sıfırdan farklı elemanların köşelerini belirlediği **dört alt kare matris** üzerinde **özyinelemeli** olarak tekrarlanır; matrisin tamamı dolana kadar sürer.
- Gürültünün ölçek çarpanı her özyineleme seviyesinde **2^0.9** kadar küçülür (`scalef`).
- ⚠️ `land(A)`'daki A, boyutu **$2^n + 1$** olan **kare** bir matris olmalıdır; sonuca **yalnızca A'nın köşe elemanları** etki eder.

Aşağıdaki örnekte (Şekil 8.19) `land` ile bir yükseklik matrisi B kurulur. Yüzeyler için **`meshz`** kullanılır: `mesh` gibi çalışır, ama yüzeyin kenarlarına **dikey bir perde** asar. 1. alt grafik B'nin varsayılan görünümü. 2. alt grafikte **"deniz seviyesi"** uygulanır: ortalamanın altındaki bütün yükseklikler ortalamaya yükseltilir (`Bisland`) ve varsayılan görünümle çizilir.

Sayfa 108 — Fraktal manzara script'i, Tablo 8.6, NaN

3. ve 4. alt grafikler `view([-75 40])` ve `view([240 65])` kullanır; bu ikisinde eksen sınırları da kontrol edilir.

Örnek 8.34 — Şekil 8.19

```
randn('state',10);
k = 2^5+1;
A = zeros(k);
A([1 k], [1 k]) = [1    1.25
                   1.1  2.0];
B = land(A);

subplot(221), meshz(B)
FS = 'FontSize'; title('Default view',FS,12)

Bisland = max(B,mean(mean(B)));
Bmin = min(min(Bisland));
Bmax = max(max(Bisland));
subplot(222), meshz(Bisland)
title('Default view',FS,12)

subplot(223), meshz(Bisland)
view([-75 40])
axis([0 k 0 k Bmin Bmax])
title('view([-75 40])',FS,12)

subplot(224), meshz(Bisland)
view([240 65])
axis([0 k 0 k Bmin Bmax])
title('view([240 65])',FS,12)
```

- `randn('state',10)` → aynı manzarayı tekrar üretebilmek için.
- `k = 2^5+1` = 33 → 33×33 matris ($2^n + 1$ şartı).

- `A([1 k],[1 k]) = ...` → sadece dört köşe ayarlanıyor (1, 1.25, 1.1, 2.0).
- `max(B,mean(mean(B)))` → iki argümanlı `max` (Bölüm 5.5): ortalamanın altındaki değerler ortalamaya çıkar = **deniz seviyesi**, düz bir "deniz" oluşur.
- `Bmin`, `Bmax` → `axis`'te z sınırları için.
- `view([a b])` → vektörle yazılmış bakış açısı.

Tablo 8.6 — En popüler 3B çizim fonksiyonları (birkaçının adı **ez** ile başlayan "kullanımı kolay" sürümleri var; bazıları Bölüm 8.3'te):

Sayfa 109 — Şekil 8.19 ve Tablo 8.6

Şekil 8.19: Fraktal manzaranın dört görünümü (varsayılan; deniz seviyeli varsayılan; `view([-75 40])`; `view([240 65])`).

Fonksiyon	Açıklama	Fonksiyon	Açıklama
`plot3` *	Basit x-y-z grafiği	`surfc` *	Dolu yüzey + konturlar
`contour` *	Kontur grafiği	`waterfall`	Tek yönlü tel kafes
`contourf` *	İçi dolu kontur grafiği	`bar3`	3B çubuk grafik
`contour3`	3B kontur grafiği	`bar3h`	3B yatay çubuk grafik
`mesh` *	Tel kafes yüzey	`pie3`	3B pasta grafik
`meshc` *	Tel kafes yüzey + konturlar	`fill3`	Çokgen doldurma
`meshz`	Perdeli tel kafes yüzey	`comet3`	3B hareketli, kuyruklu yıldız benzeri grafik
`surf` *	Dolu yüzey	`scatter3`	3B saçılım grafiği
		`stem3`	Sap (stem) grafiği

* Bu `fun` fonksiyonlarının **`ezfun`** karşılıkları da vardır (örn. `ezplot3`, `ezcontour`, `ezsurf`).

NaN = eksik veri: Bütün grafik fonksiyonlarının ortak özelliği: **NaN'ler "eksik veri" sayılır ve çizilmez** (sayfa 108).

Örnek 8.35

```
plot([1 2 NaN 3 4])
```

İki ayrı çizgi çizer: "2" ile "3"ü **birleştirmez**.

Örnek 8.36 — Ortasında delik olan yüzey (Şekil 8.20)

```
A = peaks(80); A(28:52,28:52) = NaN; surfc(A)
```

- **`peaks`** → iki değişkenli belirli bir fonksiyona karşılık gelen yükseklik matrisi üretir; 3B grafikleri göstermek için kullanışlıdır.
- `A(28:52,28:52) = NaN` → ortadaki 25×25'lik blok eksik veri yapılır (Bölüm 5'teki alt matris ataması ve skaler genişletme).
- `surfc` yüzeyi altındaki konturlarla birlikte, **ortası delik** çizer.

Karmaşık fonksiyonları görselleştirmek: MATLAB'ın **`demos`** klasöründe adı **`cplx`** ile başlayan fonksiyonlar vardır (`what demos` ile görülür).

Örnek 8.37 — Şekil 8.21

```
cplxroot(3)
```

Genel olarak **`cplxroot(n)`**, z^(1/n) fonksiyonunun **Riemann yüzeyini** çizer.

Örnek 8.38 — Demo listesini görmek

```
what demos
```

Sayfa 110 — Şekil 8.20 ve 8.21

Şekil 8.20: NaN içeren matrisin `surfc` grafiği: ortası delik dağlık yüzey, altında konturlar. **Şekil 8.21:** z^(1/3) için Riemann yüzeyi: kendi etrafında dönerek üst üste katlanan sarmal bir yüzey.

🏁 **Hocanın istediği kısım burada biter** (3B grafiklerin sonu, kitap s. 110).

Bölüm 8.2 — Akılda kalması gerekenler

1. `plot3(x,y,z)` = 3B `plot`; stil metni aynı; `zlabel`; 3B'de varsayılan `box off`.
2. `[X,Y] = meshgrid(x,y)` ızgarayı kurar; `Z` X ve Y'den **nokta operatörleriyle** hesaplanır; Z(i,j) ↔ (x(j), y(i)).
3. Kontur: `contour(x,y,Z,n)`, `ezcontour`, etiket için `[C,h] = contour(...)` + `clabel(C,h,...)`.
4. Yüzey: `mesh` (tel kafes), `meshc` (+kontur), `meshz` (+perde), `surf` (dolu), `surfc` (+kontur), `waterfall`.
5. Görünüm: `colorbar`, `colormap`, `shading flat/interp`, `hidden off`.
6. `view(a,b)`: azimut ve yükseklik; varsayılan `view(-37.5,30)`, `view(2)` tepeden, `view(3)` 3B.
7. NaN'ler çizilmez → çizgide boşluk, yüzeyde delik.